SM in the Wild: Measuring Security-Semantics Failures in Chinese Commercial Cryptography Implementations

Juwan Hwang, Shanghai Maritime University (上海海事大学).

Preprint compiled 2026-11-01 (supersedes the 2026-10-28 versions). Contact: huangzhiwen@stu.shmtu.edu.cn; ORCID 0009-0007-9231-8698. Prepared for submission to USENIX Security 2027 (Cycle 2). This is the text prepared for review; where it refers to reviewers, the Open Science appendix or a review copy, it refers to that process. Every artefact whose findings are not already public is pseudonymized (U1-U61); see Ethical Considerations.

Abstract

Chinese commercial cryptography (Guomi: SM2, SM3, SM4) exposes security inputs that international APIs and misuse taxonomies omit: the signer's identity, the curve and the ciphertext arrangement. We call this gap *contract impedance mismatch* and measure it in published software, not deployments.

We build an executable SM2/SM3/SM4 oracle validated against four GB/T standards and two Bouncy Castle defects. Over three corpora (npm, Maven, Gitee) we find 147 adjudicated rows, which reduce to **64 finding keys** (§5). Confirmation is single-author and LLM-assisted, not independent human validation, and the 144 confirmed pairs come from a re-coding of all 270 candidates after a first pass that confirmed 17 (§6.1). Every demonstrated confidentiality break but one is a generic defect (known key, fixed IV, non-cryptographic randomness), including a JCA SM4 provider that replaces caller IVs with a constant; the exception is a signature routine whose output carries the private key (F-0025). The identity, arrangement and curve categories (M3, M4, M5) yield conformance defects and permitted-but-weak defaults.

Four detector families find 9 of a curated 28-finding cohort (the first-pass v1 cohort; one member, F-0016, has since been retired), all hard-coded keys; none flags any Guomi-specific property. On 88 seeded defects our literal screen localises 37.5% and public Semgrep rule packs 10.2%; neither localises the 12 provider-contract defects. Constant screens find implementations, not uses, and a frame's membership rate must precede its precision.

1. Introduction

Empirical work on cryptographic API misuse has, with few exceptions, measured the international ecosystem: JCA in Java [1–4], Python [5], Go [6, 7] and JavaScript [8]. That is defensible for the population it studies and misleading if generalised. Here, "in the wild" means published packages and repository snapshots, not observed deployments or compromises. We take a non-Western national cryptographic ecosystem as the object, because it is where we expected a category-level gap, and because doing so tests whether the *method*, and not only the corpus, transfers.

Guomi is a good object for three reasons (§2.1). **P1:** SM2 hashes the signer's identity into the signed digest, so a library that substitutes a constant identity produces valid signatures that bind nothing, and a self-test cannot see it. **P2:** the curve is a system parameter, and providers disagree about a key on the wrong curve. **P3:** two ciphertext arrangements circulate, and the disagreement is invisible inside any single implementation.

None of the three has a slot in the APIs through which Guomi is usually reached. JCA's `Signature` takes a key and a message; it has nowhere to put an identity. JCA's `Cipher` contract assumes the provider either honours or generates an IV. A JavaScript wrapper's two-argument `encrypt(data, key)` hides a mode and an arrangement behind defaults. We call the resulting class of failures a *contract impedance mismatch*: the caller satisfies the international contract, the provider or library accepts the call and returns success, and the result is not the standardised algorithm, or binds a constant the caller never chose. Category M14 of our taxonomy (provider contract nonconformance) is its sharpest form.

We ask four questions: **Q1** what international taxonomies omit for Guomi; **Q2** which shipped defects an executable oracle confirms, at what consequence; **Q3** how to measure a conversion rate across unlike frames; **Q4** whether detectors cover the Guomi surface. Maven bears only on Q3 and the platform layer. The oracle separates surface hits from executable violations, from contract defects, from defects with

demonstrable impact. The ordering is not the intuitive one: the Guomi-specific identity, curve and arrangement categories are the novel ones and yield conformance defects, while every confidentiality break we demonstrate involves a generic property (a known key, a fixed IV, non-cryptographic randomness), sometimes reached through a broken contract.

Contributions. Two, and the second travels further than the first.

A. The Guomi contract gap, measured.

1. **An executable oracle for SM2/SM3/SM4**, calibrated on every run against GB/T 32918.5-2017 signing, 32905-2016 SM3, 32907-2016 SM4 and 32918.4-2016 encryption (§3.1).

2. **Platform contract findings.** A JCA SM4 provider ignores `IvParameterSpec` in CBC, CFB, OFB and CTR; Bouncy Castle's implicit signing identity varies by version and propagates to wrappers (§5.1). The oracle also rediscovers its known 1.57–1.58 `sign(SM3(M), ID="")` defect, fixed in 1.59 [34], not counted as new.

3. **64 finding keys** (the unit of the census; the precision figures of §6 count candidate packages; they cover 144 admitted package-level defects in 147 adjudicated rows, §5), from constant key agreement and a usable private key in decryption to fixed IVs (§5.2–5.3).

B. Methodology lessons for measuring a misuse ecosystem.

4. **Frame membership before precision** (§6.3), and a conversion rate stated with its denominator, screen and frame (§6.1–6.4), with nine precision corrections and a screening-sensitivity analysis.

5. **Implementation is not use** (§6.5): on 88 seeded defects a literal screen localises 37.5%, public Semgrep rules 10.2% and our own Guomi rules 54.5%; a recall figure is only as good as the seeding that produced it.

6. **Baselines are a rule-coverage question** (§6.6): four detector families find 9 of 28 cohort findings, all hard-coded keys.

Scope. The three corpora are different sampling frames, so no cross-ecosystem rate is compared. We do not claim population recall: there is no labelled ground truth, and the one recall figure is on seeded defects (§6.5). No finding is shown to have caused a compromise: of the eight npm findings whose dependents we searched through the original channels, one (F-0073) had a measured dependent, and a later deps.dev measurement (§7) adds manifest-edge dependents for some findings, none showing the defective path is reached.

2. Background and the Gap

2.1 The three algorithms

SM3 is a 256-bit Merkle–Damgård hash with its own initial value; those constants appear in source across the ecosystem, which makes an SM3 IV a false positive for a hard-coded-key screen. **SM4** is a 128-bit block cipher used in ECB, CBC, CFB, OFB and CTR; nothing distinguishes it from AES in shape. **SM2** is an elliptic-curve public-key system over a fixed 256-bit prime curve. A signature covers `Z_A ‖ M`, not `M`, where

```
Z_A = H_256(ENTL_A ‖ ID_A ‖ a ‖ b ‖ x_G ‖ y_G ‖ x_A ‖ y_A)
```

and `ENTL_A` is the bit length of the identity `ID_A`. The identity is an input to the hash, not an omissible field (P1). GB/T 32918 writes the curve into the algorithm definition (P2). The SM2 encryption standard and its 2012 industry-standard predecessor both specify `C = C1 ‖ C3 ‖ C2`, while `C1 ‖ C2 ‖ C3` circulates in implementations (P3).

Two properties of the standards matter for adjudication. First, compliance and security come apart: GM/T 0009-2023 [28] §10 *permits* the default identity `"1234567812345678"` "absent special agreement", so a signature bound to it is compliant and binds nothing, and a rule that flags it flags every conformant implementation. Second, part of the normative text is only available as scanned images; the one clause previously held only as an OCR transcript has since been read first-hand from the official page image (Appendix F, T7).

2.2 The taxonomy gap

JScamd [8], a recent top-venue misuse-detection work, is the closest to our npm frame. We verified its coverage from its full text (21 pages, 106,020 characters): thirteen Guomi terms occur zero times, fourteen international-algorithm terms all occur (term lists: Appendix H). Its 16 constraints include C-07 (no predictable or constant keys), C-03 (no ECB), C-05 (unique IV) and C-11 (no insecure PRNG). Five of our fourteen categories correspond (Table 1), including M1, where all of our confidentiality breaks with a shipped key sit.

The remaining categories are M6 signature nonce or weak RNG, M7 malleability, M8 bare SM3 as a MAC, M10 unchecked verification result and M13 bundled outdated cryptography. The absence in Table 1 is structural: a taxonomy built from AES/RSA/SHA usage has no slot for "the identity is inside the signature" - in those systems it is not. We carry one consequence through the paper: **the identity, arrangement and curve categories (M3, M4, M5) are absent from the taxonomies we read and yield conformance findings, while the generic categories produce the confidentiality breaks.**

Table 1. Our misuse categories against JScamd's 16 constraints. The bold rows have no entry in that taxonomy; categories not listed (M6–M8, M10, M13) map to no constraint in that taxonomy; all five do yield findings in our census (Table 14).

ours	theirs	what is new here
M1 hard-coded key or IV	C-07	category covered
M11 public value as symmetric key	C-07	category covered; the SM instance is new
M2 ECB default	C-03	category covered
M9 IV / nonce fixed or reused	C-05	category covered
M12 non-CSPRNG key generation	C-11	category covered
M3 identity binding	—	**no analogue**
M4 ciphertext arrangement	—	**no analogue**
M5 curve as system parameter	—	**no analogue**
M14 provider contract nonconformance	—	**no analogue**

2.3 Contract impedance

M14 is the case where a caller makes a legal call under a provider contract (JCA or a library's documented API), the provider accepts it and returns success, and the output is not the standard algorithm's output. The oracle criterion is byte-level comparison with a calibrated reference, or mutual verification of two API paths inside one provider. M14 is the primary category of four defects: two providers behind the JCA contract (F-0009, and F-0014 with its extension F-0018) and two npm libraries whose documented SM2 or SM4 API computes something that is not the named algorithm (F-0025, F-0028); further census keys carry M14 as one of several codes (Tables 14 and 15). The wider impedance pattern, an international API shape with no slot for a Guomi input, explains a further group: identity defaults that the JCA `Signature` interface cannot override (F-0008, F-0015, F-0002, F-0007), a curve that two providers resolve differently (F-0001), and wrapper defaults whose mode or arrangement the caller never states (F-0073, F-0006, and the `cipherMode` argument in F-0017). We use the term as an organising lens, not as a measured cause: the generic findings (hard-coded keys, constant IVs, predictable randomness) do not depend on it, and we do not count them under it. The lens makes one testable prediction - detectors whose rules attach to JCA sources and sinks miss findings on the Guomi API surface - and §6.6 measures it.

3. Oracle and Method

3.1 The oracle and its calibration

The oracle has three parts: from-scratch SM2, SM3 and SM4 reference implementations; a differential harness that runs real artefacts (published tarballs and jars, and compiled providers on a fixed JDK); and one verification script per finding, recorded for inspection against its artefact: every DEFECT row carries a recorded oracle outcome or attempt record, and two recorded PASSes stand uncredited (C.11). We did not re-verify each verdict from a clean state.

The calibration vectors follow. For SM2 signing:

```
sign(M="message digest", ID="1234567812345678", dA, k) reproduces
    r = F5A03B0648D2C4630EEAC513E1BB81A15944DA3827D5B74143AC7EACEE
    ↪ E720B3
    s = B1B6AA29DF212FD8763182BC0D421CA1BB9038FD1F7F42D4840B69C485
    ↪ BBC1AA
[k]G == (x1, y1) byte-for-byte
swapped r/s       -> verification fails
byte-reversed r   -> verification fails
DER round-trip    -> order preserved
```

This turns "is your `(r,s)` order and endianness right?" from an assumption into a tested fact [25], and makes the §5.1 matrix's control rows a rebuttal: a byte-order error would fail the controls too, since they pass through the same parse-and-verify path. SM3 is calibrated against GB/T 32905-2016 Appendix A [26], and `Z_A` against the GB/T 32918.5-2017 worked example. The SM4 implementation reproduces GB/T 32907-2016 Appendix A [27] exactly: all 32 round keys, the Example 1 ciphertext, the round trip, and the Example 2 million-iteration vector. SM2 encryption follows GB/T 32918.4-2016 and is pinned to its Appendix A worked examples; its `decrypt()` recomputes the C3 digest and rejects a ciphertext whose arrangement is wrong, which lets the oracle decide P3 questions that ciphertext length cannot (Appendix C). Four GB/T standards are calibrated in total. No oracle path a verdict relies on is uncalibrated (named exceptions rest on recorded third-party or historical evidence, C.11); auxiliary paths carry stated limits and support no verdict (Appendix C). The residual limits are in standards reading and downstream measurement (§7).

The oracle takes no Guomi library as ground truth. This matters: the npm library `sm-crypto` had an SM2 signature-forgery flaw before version 0.4.0 (CVE-2026-23965 [24]). It only causes false acceptance of crafted signatures; we nevertheless re-ran every verification our npm probes report against 0.4.0, and every verdict is unchanged. Outside browser environments, the same library before 0.5.0 seeds SM2 key and nonce generation from a jsbn SecureRandom pool the wall clock alone fills (CVE-2026-73567, GHSA-vh45-f885-3848 [35]): jsbn 1.1.0 reads `rng_psize` before its `var` assignment, so the Math.random fill is dead code and, where `window.crypto.getRandomValues` is unavailable (Node), `Date.getTime()` alone seeds the pool. We reproduced zero `Math.random` calls in Node; the advisory's first text (and NVD, as of 2026-11-02) said Math.random-and-clock, and the maintainer's repository advisory was corrected to clock-only on 2026-10-27 [35]. The 0.5.0 fix (commit 1f9bd7bd) replaces jsbn's generator; it cannot reach the vendored copies our rows measure (§5.2).

3.2 The pipeline

```
N1  discover     registry search · reverse-class lookup ·
↪  repository search
N2  obtain       tarball · sources.jar · shallow clone
↪  (cached, resumable)
N3  screen       literal misuse patterns ∪ structural (algorithm
↪  data) ∪ use screen
N4  triage       publishability (repository / maintainer /
↪  private) · severity class
N5  confirm      executable oracle, on the SHIPPED artefact where
↪  one exists
N6  adjudicate   human verdict, only for what survived N5
```

N3 matches the shipped artefact's own code, because bundling is a measured false-positive class (33 of the 83 Maven jars only bundle a third-party SM library): npm screening skips `node_modules`, Maven screening reads each artifact's own sources, and all frames skip `vendor/`, `test/`, fixtures and `.d.ts` through one shared file scope. **N5 runs the published artefact**, since only "the released artefact has the bug" is disclosable. **N6 adjudicates oracle survivors**; the three exceptions - one read-only review, one never run, and one whose shipped default path was never run though its arithmetic was checked (C.12) - were adjudicated defects on their records. N5/N6 is 100% on npm (124/124) and 87.0% on Gitee (20/23), and is never reported as precision.

3.3 What counts as a finding

A finding is admitted only if four conditions hold: it is **executable** (recorded executable evidence reproduces it: a script, or a recorded run with its output; 140 of the 144 confirmed pairs replay under the recorded replay, §3.1); it is **on a shippable artefact** (a published package or provider, or a repository that is itself the object); it has a **stated boundary** (what the oracle proves, what it does not, what needs human adjudication); and it has a **negative control** (the effect disappears when the suspected cause is removed; for a shipped key, where nothing can be varied, the check that the code path works with an independent key pair, or that `[d]G` equals the shipped public key, plays this role). The control is what separates "this code looks wrong" from "this code is wrong": for a constant IV, changing the IV removes the leak; for F-0009, changing the version changes the behaviour. Admission rests on the recorded evidence; §5 states how uniformly it survives re-run.

We grade evidence on a ladder used throughout the finding records: L1 and L2 are conformance-level observations (a behaviour that diverges from the standard or between implementations, without a violated security property); L3 is a confirmed misuse in shipped code whose consequence is not demonstrated; L4 is a violated security property confirmed on the shipped artefact; L5 adds a demonstrated consequence or closed usage chain (decryption from public data, a usable private key on a live path). The precision figure counts findings at L3 and above in the application-layer frames; library and platform findings are reported separately (§6.1).

3.4 Threat model

We grade consequences against three attacker capabilities, and every finding states which one it needs. A *reader* has the published artefact and captured ciphertexts or signatures, which is enough for hard-coded keys, constant keys and public values used as keys (F-0003, F-0012, F-0013, F-0021, F-0024). An *observer* additionally sees repeated encryptions or some outputs of the same process: equal ciphertexts under a constant IV (F-0010, F-0014), or `Math.random()` outputs read in the same JavaScript isolate (F-0004, F-0017). A *peer* is any other holder of the same library, which suffices for keys that are identical across processes or installations (F-0021, F-0026). We do not consider side channels, faults, or attacks on the primitives themselves; we treat a conformance divergence as a security finding only when one of these attackers gains something from it, which is why F-0001, F-0008 and F-0009 are not graded as vulnerabilities. We never probed deployed systems, so where a finding needs a deployment to become an attack, it stops at the artefact.

4. Measurement Design

4.1 Three frames, no cross-frame rate

Three corpora, three sampling frames, and no rate is compared between them (Table 2). The npm corpus samples packages that publish Guomi; the Gitee corpus samples repositories matched by name or description; the Maven corpus samples artifacts whose dependency closure contains an SM class. Per-package and per-repository defect rates have different denominators and no cross-ecosystem reading; we report a per-frame table (Table 7) and each frame's *membership rate*, the share actually carrying Guomi (§6.3).

The npm corpus was built in two waves (413 packages from 11 terms, then 381 from 39 more terms and reverse-dependency trails; term records: Appendix E); Gitee's search returned 404, so its population was reconstructed by reverse lookup.

4.2 Where the funnel inflates

Maven's `fc:<class>` search returns artifacts whose dependency closure contains the class; Bouncy Castle ships `SM2Signer`, so every BC-dependent artifact matches, and 333 artifacts whose closure contains an SM2 class reduce to one that uses Guomi in its own source - measuring Bouncy Castle's popularity, not Guomi use.

4.3 Three screens

N3 is the union of screens that work on different principles, run over one shared file scope (all three in npm; the first two in Maven and Gitee):

Table 2. The three sampling frames, and what each can and cannot answer.

corpus	population	frame	answers	cannot answer
npm	794 packages from 11 + 39 search terms	published-package sample	application-layer misuse in shippable packages	packages that vendor Guomi without publishing
Maven Central	83 artifacts whose dependency closure contains an SM class	reverse-class sample	library and platform behaviour	application-layer behaviour — the frame selects dependents, not users
Gitee	1415 discovered, 59 cloned	repo-name/description search	source patterns in the Chinese ecosystem	dependency relations — Gitee has no code search

- a **literal screen** of ten misuse patterns (M1–M13 plus arrangement), with a Java variant for Maven and a multi-language variant for Gitee;
- a **structural screen** that detects Guomi by algorithm *data* (the SM4 S-box, the SM3 initial value, the SM2 curve parameters) in 46 encodings across six languages and Vue single-file components, held to a 15-table negative control;
- a **use screen** that detects Guomi by *use*: an import, `#include`, or call of a Guomi library symbol.

The second and third screens were added because the first was measured to miss (§6.5). Every N3 candidate carries a verdict, individual or by class, anchored to a shipped-artefact substring re-checked on every run (Appendix D).

4.4 Confirmation and what we do not measure

An N5 confirmation requires, in order: the behaviour observed on the published artefact where one exists; a negative control; the affected version range, distinguishing behaviourally tested from structurally checked versions; and, for anything to be disclosed, a usable channel (Ethical Considerations; Appendix G). We do not measure population recall (no ground truth) or real-world compromise (§3.4).

5. Results

The census admits 144 package-frame pairs, grouped into 64 finding keys; 27 keys reach L5 (Table 3). They come from 147 adjudicated rows over 270 screen candidates. Three adjudicated rows are not counted because the shipped artefact never reached the oracle (Appendix C.12). Recorded replay supports 140 of the 144 pairs; the other four rest on weaker records that we state with them (Appendices C.7, C.10, C.11). Confirmation is single-author and LLM-assisted, not independent human validation. The platform and library observations of §5.1 are narrated separately and are not part of this census. Packages whose maintainers are still inside a disclosure window appear as U1-U61 (Ethical Considerations).

Two sentinel keys group 17 L3 packages (15 npm, 2 Gitee). Each has an internal adjudication row with locus, reproduction and negative control, but the evidence is not reviewer-accessible (Open Science) and five of the 17 probes cannot be mapped to an archived script. No L4 or L5 count rests on them; without them the census is 127 pairs, a sensitivity and not a remeasurement. Removing the 54-package F-0032 grouping as well leaves 73 pairs in 61 keys, 26 of them L5. Three of the 27 L5 keys (F-0007, F-0038, F-0040) need an observer of the signer's stdout or logs (§3.4); F-0012, F-0021 and F-0024 need only a reader.

5.1 Platform and provider findings

Four platform-level results: a known Bouncy Castle defect that our oracle rediscovered (the validation case); identity defaults that no caller can override; two providers that resolve a wrong-curve key differently; and a JCA cipher that ignores the caller's IV.

F-0009: a known provider defect, rediscovered (validation case). `bcprov-jdk15on` 1.57 registers `Signature.SM3withSM2`, but its SPI hands the SM3 digest of the message to a signer that expects the message itself. The JCA signature is therefore `sign(SM3(M), ID = "")`, a standard SM2 signature of the prehash under an empty identity, and not `sign(M, ID = default)`. Bouncy Castle already knew: bc-java #218 (2017) reported both the repeated hashing and the missing default identity, and 1.59 fixed them [34]. Our oracle reached the defect without that history, which is the check we wanted: the method finds a real defect. Version 1.57 still resolves in our corpus, through `bcpkix-jdk15on:1.57` (U18, F-0007). In the verification matrix of Appendix A.1 the 1.57 and 1.58 JCA signatures verify 32 of 32 under `(SM3(M), ID = "")` and 0 of 32 under every reading that uses M, and a bisection over ten further jars puts the change at 1.59 (`SM3withSM2` is absent in 1.55-1.56).

This is the impedance mismatch in its sharpest form: a DSA-shaped SPI written for digest-then-sign algorithms, wrapped around a signer whose pre-hash includes `Z_A`; the private key is used correctly, there is no forgery surface - an interoperability defect, not a vulnerability.

F-0008, F-0015, F-0002, F-0007: identity defaults the API cannot override. `SM2Signer.init` takes the identity from a `ParametersWithID` wrapper and falls back to a constant when it is absent, and the constant depends on the version (Table 4). The same call under two dependency versions binds two different identities; the trigger is an ordinary dependency

Table 3. The census by standing: 64 finding keys over 144 package-frame pairs. L3 is misuse in shipped code, L4 a security property violated on the shipped artefact, L5 a demonstrated consequence; a key takes the highest level among its packages. Full census: Tables 14 and 15 (Appendix C.7).

standing	npm	gitee	gitee+npm	keys
L5	18	8	1	27
L4	21	7	-	28
L3	7	2	-	9
all	46	17	1	64

upgrade. The bisection above places this switch at 1.59, where Bouncy Castle fixed the empty default it called wrong [34]; F-0008's empty-ID half is known.

The default reaches APIs developers call. Measured with the same oracle, `hutool-all:5.8.35` (bundling bcprov 1.66) signs under the default identity when none is given and honours a real identity when one is. The measurement works because `Z_A` is observable: the identity a library bound is identified by which candidate `Z_A` its signature verifies under. On the JCA path the identity cannot be supplied at all: `U44`'s SM2 signature class hard-codes `"1234567812345678"` for signing and verification, `engineSetParameter` is a no-op, and a signature validates only under the default identity (F-0015). `litesdk` exposes an identity parameter that callers can omit (F-0002); `U18` exposes none (F-0007).

Because GM/T 0009-2023 §10 permits the default identity "absent special agreement", we report F-0002, F-0015 and F-0008's 1.59+ default as observations, not defects: the binding is *weakened*, the standard is not *violated*. §10 names no empty identity, so the 1.57–1.58 half is a known conformance defect [34], interoperability-only, and not counted as new. The category nevertheless splits in a way that matters for remediation: M3-L1, the caller could pass an identity and did not (F-0002, hutool), is fixable at the call site; M3-L2, the library offers no identity parameter (F-0007, F-0015), is fixable only in the library. "Change the default" would mask both.

F-0001: one wrong-curve input, two curves. P2 is observable. We gave two providers a public key genuinely on `secp256r1` together with the caller's private scalar, and replayed both signatures through our verifier under two hypotheses: H1, signed on the curve of the supplied key; H2, signed on the SM2 curve using the caller's scalar (Table 5).

Kona produces something that is not an SM2 signature under the supplied key and then refuses to verify it. `hutool-all-5.8.35` is no third data point: its `SM2.class` references Bouncy Castle's `SM2Signer` and `SM2Engine` directly (bytecode-verified). Neither behaviour is a bug in isolation; the defect is the disagreement, invisible inside either implementation, and we grade it L2.

F-0014 and F-0018: a JCA cipher that ignores the caller's IV. `U44`'s `U44`'s provider registers an SM4 `Cipher` whose default mode is CTR. Its `engineInit` accepts only its own `SM4AlgorithmParameterSpec`; a standard `javax.crypto.spec.IvParameterSpec` is dropped without exception, and `engineDoFinal` substitutes `"1234567812345678".getBytes().engineGetIV()` returns an empty array while an IV is in use, and `engineGetParameters()` returns `null`. On the published jar (SHA-256 `4A329E99...3A37`), a caller-supplied IV gives the same ciphertext as no IV, two different IVs give the same ciphertext, and with a constant IV in CTR two encryptions under one key satisfy $C1 \oplus C2 = P1 \oplus P2$; a different IV removes the relation. F-0018 extends this to CBC, CFB and OFB, and our own SM4, given the constant IV, reproduces `U44`'s CBC output byte for byte. Table 6 rules out the defence that "SM4/CTR behaves this way": Bouncy Castle's SM4/CTR on the same JDK generates and reports a random IV.

The `CipherSpi` contract expects a provider to generate missing parameters and make them retrievable, or to refuse [30] (Appendix A.3); this provider does neither. Every published version contains the affected classes, behaviour is identical on the three we ran, and there is no fixed version. The same artefact carries F-0019, an encrypt/decrypt default that disagrees (decryption silently substitutes the constant IV that encryption refuses, yielding wrong plaintext without an error; a correctness defect), and F-0016, a live hard-coded private key in its TLS server class with no caller found.

5.2 The application layer

The 144 confirmed pairs fall into a few recurring mechanisms, described below; Appendix C gives each finding's evidence.

Key agreement that returns a constant (F-0021). `U47`'s main class module performs a genuine Diffie-Hellman exchange and then returns, as the SM4 key, a compile-time literal (`0123456789abcdeffedcba9876543210`) derived from no input. The library's own `decrypt` with that constant recovers the plaintext, and the README documents passing the returned key to `decrypt`, so the intended protocol is the broken one. The DH exponent is also recoverable (99,800 values, 169 ms), but the ciphertext falls without it. It is a library defect; none of our 794 packages or the author's seven other packages depends on it.

Public values and predictable randomness as keys (F-0003, F-0017, F-0004, F-0026). `U46` uses the first 32 hex characters of the SM2 public key as the SM4 key, so plaintext follows from public data alone (F-0003); the published package cannot be `require()`d because of a packaging error and had few downloads in the year to 2026-10-12, so the affected deployments may number zero. `U45` builds its hybrid-encryption session key from 16 `Math.random()` draws (F-0017), and `U3` does the same (F-0004). V8's xorshift128+ is

Table 4. The implicit signing identity moves between bcprov versions.

bcprov	fallback	meaning
1.57–1.58	`userID = new byte[0];`	empty identity
1.66 / 1.78	`userID = Hex.decodeStrict("31323334...3738");`	the standard's example identity `"1234567812345678"`

Table 5. One wrong-curve input, two providers, two resolutions.

implementation	supplied key is on	H1	H2	verifies its own signature
BouncyCastle 1.78	`secp256r1`	**yes**	no	yes
Tencent Kona 1.0.17	`secp256r1`	no	**yes**	**no**

Table 6. The negative control for F-0014: the same `init(ENCRYPT_MODE, key)` call without an IV on Temurin 17.0.20.

provider	cipher	`getIV()`	key reused
SunJCE	AES/CTR/ NoPadding	real IV	new ciphertext
BouncyCastle	AES/CTR/ NoPadding	real IV	new ciphertext
BouncyCastle	**SM4/CTR/ NoPadding**	real IV	new ciphertext
U44 provider	**SM4**	**empty array**	**same ciphertext**

linear, so its state is recoverable from observed output. Our GF(2) solver, which also models V8's 64-entry cache, predicts the next session key from eight `Math.random()` outputs read in the same isolate in 20 of 20 trials per package, 40 of 40 across the two. The grade is therefore *exploitable under same-isolate observation*; without an observed output the chain does not apply. `U55` seeds an HMAC-DRBG with constants, so the first SM2 private key generated in every Node process is the same scalar (F-0026).

Shipped keys: usable versus present (F-0012, F-0013, F-0024 versus F-0022, F-0011). Five findings publish SM2 private keys, and one computable property separates their grades: whether `[d]G` equals the published public key. `U4` publishes two self-consistent SM2 pairs and two SM4-CBC key/IV pairs in a class named `EncryptionStaticKey`, used by its transport, database and configuration utilities (F-0012). `maku-boot` (36,194 stars at the corpus scan) decrypts the submitted login password with a self-consistent hard-coded private key, and its frontend hard-codes the matching public key (F-0013); a third party reported the same key publicly on 2026-08-13, so we claim independent verification of the pair and the closed source chain, not discovery. `U48` passes a self-consistent hard-coded private key to `doDecrypt` on its live path (F-0024); the same pair, SM4 key and IV sit in the widely starred (same scan) `U49` server as `DATA ^ MASK` byte arrays reconstructed at class load. The server uses them to decrypt passwords at login, registration and reset (10 call sites), to encrypt nine identity and phone columns and TOTP seeds (25 call sites), and to sign audit logs. A public audit had flagged an older plaintext version of this key in the server; the XOR split that followed removes the hex literals and adds no security, since the pair was not rotated. It is a fix aimed at scanners, and it works on ours (§6.5). By contrast, the pair shipped by three npm packages does not correspond (F-0022, L3), and `U57`'s inconsistent pair makes its default decryption silently return an empty string (F-0011). An earlier write-up of F-0011 as "decrypt with public data alone" was withdrawn for exactly this reason.

Silent defaults (F-0023, F-0029, F-0006, F-0010). `U52` falls back, with no warning, to a published SM4 key whenever the caller omits `sm4Key`, and its request interceptor then encrypts every outbound body under it (F-0023); the constant is byte-identical to F-0021's, which suggests a value copied from example code. The same constant also sits in `U53`'s source, but there the fallback is dead in the normal flow - the module's setup hook always installs a key and IV before enabling encryption - so the fallback claim in this paragraph belongs to `U52` alone (Appendix C.4). `U51` has the same shape (F-0029). `U1` calls its wrapper's two-argument `encrypt`, which defaults to ECB; 64 identical bytes encrypt to four identical blocks (F-0006). GM/T 0009-2023 §7.4 requires SM4-ECB for SM2 key wrapping, so the finding holds only if the data is business data, not a wrapped key, which the oracle cannot tell. `U2` hard-codes an SM4-CBC IV with its validation commented out: equal plaintexts give equal ciphertexts and a shared 16-byte prefix gives an identical first block, both removed by a different IV (F-0010) - a dictionary and prefix leak, not the keystream reuse of F-0014.

Arrangement as an API choice (F-0073). U5 defines an enum with both arrangements, and across all eleven published versions all 22 call sites select the non-standard order; a caller cannot obtain the standard order through its API. Two same-vendor manifest edges pin the defective version in the deps.dev index (queried 2026-10-26): one released application, whose call sites carry the defect, and one release candidate (reachability confirmed statically, Appendix C.3); the latest published version lists none. GB/T 32918.4-2016 and GM/T 0003.4-2012 [29] both write `C1‖C3‖C2`, while the in-scope libraries expose `C1C2C3` as a selectable mode (Table 12, Appendix C.3), so which arrangement a deployment follows is a conformance judgement; the arrangement claim's L4 is the original adjudication, re-graded to L2 (Appendix C.9); the key stands at L5 on its newly evidenced M1 (§5). Exposing the choice is not a defect; a hard-coded non-standard arrangement classifies as M4, its security property undemonstrated.

Table 7. Use-screen adjudication, split by whether the candidate was already in N3. Verdicts as adjudicated on 2026-10-23; Table 9 counts N5 packages in the later, larger census, so its use-screen-only count (10) is not comparable with the 3 here.

	already in N3 (26)	increment (40)
defect, confirmed on the shipped artefact	**4**	**3**
defect in source, shipped artefact cannot run it	1	0
conformance defect, not security	0	1
below the L3 threshold / mention only	0	4
clean	21	27
not Guomi	0	5

5.3 What a third screen found that the first two cleared

The use screen found five findings the first two screens had cleared.

All 66 use-screen candidates were adjudicated, each verdict anchored to its shipped tarball. 26 were already structural candidates; 40 are the screen's increment and all 40 enter N3, including five non-Guomi false positives (a sound player, a string map), which belong in the screen's denominator. Table 7 gives the verdicts.

The seven confirmed defects become five findings. Two are implementations that are not the algorithm they are named for: `U54`'s SM2 signature is the private key concatenated with SM3(m), so any signature leaks the key (F-0025); the `U58` family writes SM4's rotation with JavaScript's arithmetic `>>`, so the GB/T 32907 vector encrypts to `314e6bc7...` instead of `681edf34...` (F-0028, three packages, by F-0022's precedent of one finding per shared defect). One is a predictable key (F-0026, above). Two are shipped keys: a constant SM4 key split into eight chunks and permuted back in `U56` (obfuscation aimed at a literal screen ours indeed missed, F-0027), and a silent default in `U51` (F-0029). `zhilv-sm4-gm-crypt` derives key and IV from the same 16 characters, but all three published bundles throw before producing a ciphertext, so it is not counted: N5 is defined on the shipped artefact.

The four confirmed defects among the 26 already-counted candidates had been cleared by a class verdict ("carries SM2 by data, names no misuse category"). None trips a misuse pattern: the defect is in the implementation, not at a call site, and a class-verdict clearing ground cannot see a broken implementation (§6.5).

5.4 Per ecosystem

Table 8 summarises what each frame established. The Java contribution is at the library and platform layer: in the Maven frame, after filtering to artifacts whose own source names an SM algorithm, 13 candidates reduce to one, `litesdk` (F-0002), which is a library and outside the numerator. Maven is thus a negative result about the sampling frame, not a third-ecosystem census. This is a property of the frame (§6.3). Classifying the 228 repositories Gitee discovery returned with Java as the main language shows they do: 80 are deployable applications holding 70,151 of the frame's 73,310 stars (same scan), 63 of them reaching Guomi through a declared dependency (its recorded 30-sample manual check agreed on 28, both errors toward application, so the record treats 80 as high-side, about 75). The two strongest key findings in the study, F-0013 and F-0024, are in that application layer. The published Gitee frame is the capped one: lifting the eight 600-file caps adds five candidates and no confirmed row, and the `U49` server entered only through its mobile client's matching pair (§6.5).

(Gitee N6 exceeds N5 by 3: three rows were adjudicated defects without the shipped artefact reaching the oracle - one read-only, one never run, one source-read with arithmetic only; precision uses N5 only.)

6. Measuring the measurement

6.1 The funnel and the precision figure

Figure 1 and Table 8 give the per-frame funnel with the steps any precision figure depends on.

The first-pass N5 was 17 of 270 (6.3%). The 144 comes from re-coding all 270 candidates against the full M1-M14 checklist with rules added after that pass (Appendix C.11); the two figures measure different definitions, and the first-pass figure is preserved. The reconciliation from the 152 DEFECT rows to the 147, 144 and 140 counts is laid out in Appendix C.9.

The precision figure is N5/N3 - the abstract's candidate-to-confirmed-defect conversion rate; we keep the shorter term. On npm it is 124/219 = **56.6%** over the full corpus; no three-frame rate is reported (§4.1). The numerator is a list with the three exclusions stated in §5. Confirmation is within-measurement and not uniform (§5), not independent external validation: the oracle is standards-calibrated (§3.1), but screens, probes and adjudication are single-author and LLM-assisted (§7). One finding key dominates the numerator: F-0032, a recorded adjudication grouping of 54 weak-entropy (M12) rows, contributes 54 of the 144 (54 of npm's 124); it is a topic grouping, not a verified single-origin family (Appendix C.8). Its 42 L5 rows are the jsbn 1.1.0 SecureRandom-hoisting construct also behind CVE-2026-73567 in `sm-crypto` (§3.1; [35]): for them the census measures how widely one known construct propagated, and at key level it is counted once. The other twelve rows are L4. Counting scenarios, not re-estimates of the defect rate: clearing the grouping moves npm precision to 70/219 = 32.0%, removing it from both sides to 70/165 = 42.4%. Each package is a separately shipped artefact, individually adjudicated and confirmed under the §3 rule; the grouping is a unit-of-counting choice the reader can re-divide.

Table 8. The funnel per ecosystem, with both steps that any precision figure depends on.

stage	npm	Maven Central	Gitee
N1 discovered	794 (11 + 39 search terms)	83 (closure contained an SM class)	59 (of 1415 discovered)
N2 obtained a shippable artefact	794 / 794	82 / 83 sources.jar	59 / 59 of the 60 scanned
N3 at least one screen candidate	**219**	13	38
N4 publishable	115 / 115 literal candidates; not used as a filter for the 104 re-screen candidates, each adjudicated directly	—	—
N5 confirmed (§5)	**124**	0	20
N6 adjudicated a defect	124	0	23

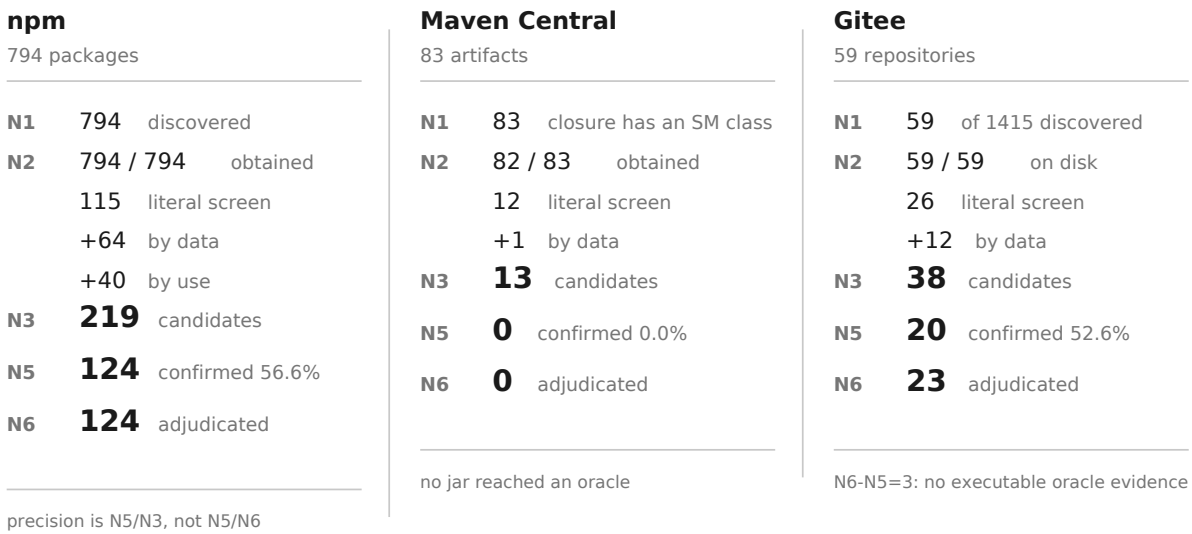


Figure 1. The funnel per frame (N1 discovered, N2 obtained, N3 screen candidates, N5 confirmed at L3 or above (§5), N6 adjudicated; N4 triage removes nothing counted here). Precision is **N5/N3, never N5/N6**: the near-100% at N6 (144/147 overall; only Gitee has non-survivors, 20/23) is an artefact of adjudicating almost only what survived an oracle. Frames are drawn side by side and not compared.

Two properties of the denominator matter. It was **re-screened in full** (§6.5): the zero-scored part of the population was screened in full by the structural and use screens, which made precision worse before adjudication made it better. And it has **no unopened candidates**: every one of the 270 carries an individual verdict or a class verdict (Appendix E).

6.2 One figure, three populations

The pooled npm figure (56.6%) hides a spread from 73.9% to 25.0% by screen.

npm's N3 is a union of three screens that name different kinds of artefact, so we report the cross-tabulation (Table 9). The literal screen names applications whose call sites match a misuse pattern; the structural screen names artefacts that vendor an implementation; the use screen names artefacts that call or import a Guomi library by a symbol.

The use screen converts lowest, and the literal screen - which names call-site behaviour - converts best. The structural half's 29 confirmations are not call-site misuse at all: each is an implementation that is itself wrong (the largest group is the recorded F-0032 grouping, Appendix C.8). An implementation is not a misuse site, but it can be a defect. We publish the pooled figure; 73.9% is N5 over a different population. The general rule: where a candidate set is a union of screens working on different principles, report the cross-tabulation, because the pooled number's meaning depends on the halves being exchangeable and these are not, by construction.

With no ground truth, the rate is not an estimate of how many defects exist. For other researchers it is a planning figure: the adjudication effort a screen of this kind costs per confirmed defect in a given frame, and which screen to spend it on.

6.3 Membership before precision

Precision alone makes three different situations look alike. We measured, with the same structural detector, how much of each frame actually carries Guomi in its own source (Table 10).

(The Gitee row counts the 60 scanned repositories, one of which could not be cloned and is counted as not carrying Guomi; Appendix I re-measures it. The Maven row counts the 82 frame artefacts with a jar on disk; the frame selects 83, and the one without a jar contributes nothing extractable.) The Maven frame selects artifacts because something in their dependency closure contains an SM class, so 81 of 82 carry no Guomi of their own, and 12 of its 13 candidates are not Guomi code at all: ECDSA nonce sources, Bitcoin verification, Bouncy Castle's own `bcpkix`/`bcmail`, Hyperledger Fabric, an OAuth2 AES key. Its 0.0% measures the share of unrelated cryptography in a Guomi frame. Two methods that share nothing, constant-pool reference analysis of the bytecode and constant fingerprints in the sources, independently reduce the frame to the same single artifact (F-0002). A frame pointed at the wrong thing produces a precision that is arithmetically correct and substantively empty. **The rule: report a frame's membership rate before its precision.** It applies to any sample defined by retrieval, citation or dependency.

The same measurement suggests dividing by on-topic candidates only, which would raise precision. We do not: an off-topic candidate is not a cleared candidate, and the one time this project measured the assumption that unread candidates are clean, it was wrong (§6.5). The bound is recorded in Appendix E, used nowhere.

6.4 Corrections and false-positive classes

The literal screen went through nine precision corrections (Table 12, Appendix B.5). Six of the nine fixed a pattern that treated correct API usage as a defect, the failure mode rule-based detectors are most prone to and least likely to report.

Corrections 7-9 were found by failing unit tests, not inspection: their output, a plausible-looking hit, is indistinguishable from a real one. We classify four measured false-positive classes:

1. **Correct API usage**: RFC 2104 key shortening, SM2 X/Y coordinate splitting, HMAC-SM3, `crypto.randomUUID()`.
2. **Documentation**: `.d.ts` declarations, JSDoc and Javadoc examples.
3. **Test fixtures**, such as an M1 hit in `test/sm4.test.js`.
4. **Unrelated secrets and non-Guomi code**: a music-service API secret, an EVM L2 project, a same-named artifact under another groupId.

A fifth source is a sampling-frame error, not a pattern error; §6.3 measures it.

A second coder. A stratified sample of 55 npm candidates (19 positive, 36 cleared), blind to our verdicts, was coded from the written rules by an LLM agent - not a human rater, and not blind to this paper (it named 18 at the time). Cohen's κ was **0.84** (bootstrap 95% 0.67–0.96; 0.54 on the 37 items unnamed), 51 of 55 items agreeing, against our first-pass (v1) verdicts: a check on rule decidability, not on the 144 re-coded pairs. Of its four disagreements, two were our errors (F-0012 artefacts; N5 unchanged), one a demo widget, one F-0021; against the revised key 53/55 agree and κ is 0.92, which is not independent. Six further LLM passes gave κ 0.31–0.66 and surfaced six real misuses (seven packages) the literal half had cleared on the M3 rule alone (App. F, T11).

Table 9. npm precision by the screen that named the candidate.

half	what it is	N3	N5	precision
literal screen	an application-layer behaviour in the shipped artefact	115	85	**73.9%**
structural screen only	an artefact that vendors a Guomi implementation	64	29	**45.3%**
use screen only	an artefact that calls or imports Guomi, by a symbol	40	10	**25.0%**
pooled	what N5/N3 is defined over, and what this paper publishes	219	124	**56.6%**

Table 10. Frame membership rate beside frame precision.

frame	artefacts counted	carry Guomi in own source	membership	N3	N5	precision
npm	794	151	**19.0%**	219	124	56.6%
Maven Central	82	1	**1.2%**	13	0	0.0%
Gitee	60	30	**50.0%**	38	20	52.6%

6.5 What the screens miss

Every class above is a false positive. This section measures false negatives: the structural screen found Guomi code in one in six packages the literal screen had scored zero (15.8%; a screening disagreement, not a count of missed defects), and the literal screen localises 37.5% of seeded defects.

Sensitivity of the literal screen. A minifier rewrites identifiers and folds constants, so the strings a literal pattern matches disappear while the code stays; on a known-positive package whose Guomi code is present in three of its four published outputs, a literal-signature screen reports it in none (Appendix B). We built the structural detector and drew a seeded random sample of 60 packages the literal screen scored zero (57 fetched). Nine carry Guomi by data, each confirmed by reading the constant in context: a screening-disagreement rate of **15.8%** (n = 57, Wilson 95% 8.5%–27.4%). Rather than extrapolate, we re-screened the whole zero-scored population, which added 64 structural candidates and moved N3 against us. Adjudicating them produced F-0021, F-0025 and F-0028. A holdout of four public advisories in one library (`sm-crypto`) is a negative result for the original pipeline: it nominated all four (fixed versions equally), localised none, and its oracle confirmed one; probes written after reading the advisories then reached four of four, which is not a holdout result for those probes (Appendix C.11).

Constants detect implementations, not uses. The structural detector decides that an artefact carries Guomi by finding algorithm data, so it measures *vendoring*. A wrapper that delegates to a Guomi library carries no such data. On twelve artefacts that are Guomi by use and carry no algorithm data, the structural screen fires on **0 of 12** and the literal screen on **12 of 12** (the per-artefact record was not preserved, so this count is carried as reported). A later re-measurement over the 794-name screened corpus reproduces the mechanism: 56 of 140 use-positive artefacts (40%) carry no detectable algorithm data; the literal screen names 16 of them, and 40 are visible only to the use screen. In an application-layer study the wrappers are the subject, not the noise, which is why we built the use screen and adjudicated all of its 66 candidates (§5.3). The rule generalises to any detector that recognises cryptography by its constants: its recall on the population it is for must be measured, not assumed.

Class verdicts cannot see broken implementations. Before the use screen, class verdicts about misuse patterns covered all 86 candidates without an individual record, and we had concluded that nothing was cleared merely because the screen saw nothing. That was wrong for four candidates: the broken implementations of §5.3, which no misuse pattern names. With the earlier sweep of Appendix E, this is why the paper divides by every candidate.

Blind spots in our own screen. An identifier rule matched `privateKey` but not `PRIVATE_KEY`, missing F-0013; its repair briefly matched 56 of 59 Gitee repositories; the shared file scope omitted five languages (Appendix B). Two evasions are adversarial: the permuted key of F-0027 and the XOR split in the `U49` server, on which our literal key rule fires zero times. The server also exceeded the 600-file per-repository cap and entered only through its mobile client's matching pair, a false negative of the screen. Lifting the cap on the eight repositories that exceed it adds five candidates and no confirmed row, which moves Gitee precision from 52.6% (20/38) to 45.5% (20/44). We publish the capped frame and report this as sensitivity (candidate-level detail: Appendix I).

Recall on seeded defects. Into four published npm packages (three compliant wrappers and the `sm-crypto` they call) we seeded 88 single-edit defects in eight categories, each written four ways: spelled out, split or renamed, behind a default, and minified. Every clean host is already a candidate, so nomination is inherited and is not recall; we score *localisation*, a new hit of the seeded category against the clean file. The literal screen localises **33 of 88** (37.5%, Wilson 28.1%–47.9%): 18 of 22 spelled out, 11 of 22 minified, 4 of 22 behind a default and **0 of 22** split, the evasion of F-0027 and `U49`. No style localises any of the 12 M14 defects. Semgrep, configured as in §6.6, flags 9 of 88 (10.2%), all M1. The category-blind oracle confirms all 88, firing only the seeded category except on the four `sm-crypto` M12 key-generator defects, where it also fires M6 (the seeded generator also seeds the signing nonce), and fires on none of ten clean controls; as we wrote both, this bounds execution from above.

Seeded Java defects. In one compliant Java host, `gmhelper`, we seeded 24 defects in seven categories and three styles. Both literal screens localise **3 of 24** (12.5%, Wilson 4.3%–31.0%); Semgrep flags 1, SpotBugs/FindSecBugs 4, CogniCrypt none. Together they reach 7 of 24, none of the 12 M2, M4 and M14 defects; the oracle confirms all 24.

Saturation, stated narrowly. Doubling npm discovery from 413 to 794 packages added no finding under the literal screen, and three rounds of pattern correction removed whole false-positive classes; the two confirmed findings keep their hits in every committed post-correction snapshot. Saturation holds for discovery, not for screening: on the same population, the structural and use screens added six npm findings.

6.6 Baseline detectors

Would off-the-shelf tools have found these defects? We ran four tool families, unmodified, over the curated 28-finding cohort: Semgrep OSS 1.182.1 [32], SpotBugs 4.9.8 with FindSecBugs 1.14.0 [33], CogniCrypt_SAST 5.0.1 with the published CrySL JCA rules [2] and CryptoGuard 04.05.03 [3] (configurations and applicability: Appendix J). Together they detect 9 of 28 (32%; v1 cohort, F-0016 since retired): all nine are hard-coded keys, the tools' home ground; Guomi-specific properties (M3, M4, M5, M11, M14) draw zero detections, as do the generic non-key properties (Table 19, Appendix J). None of these tool configurations has Guomi rules (the public CrySL repository has no SM rule), so the misses follow from rule coverage, not from the contract's shape.

Does a Guomi-aware rule set close the gap? A limited test says it closes part of it. We wrote nine Semgrep rules for the Guomi categories (constant key or IV, ECB default, default identity, C1C2C3 order, public value as key, non-CSPRNG, SM3 sign-extension, double hash) in one pass, with one syntax-debugging round and no tuning against results. On the 88 seeded defects they localise **48 (54.5%)** against 9 for the public packs: 19 of 22 spelled out, 16 minified, 11 behind a default, 2 split. They localise 5 of the 12 M14 defects (sign-extension in all four styles, double hash once, in split form; none of the signature-embedded keys) and fire on the compliant hosts too (3 to 12 hits each on three of the four clean packages), so these hits are not precision. Both the defects and the rules are ours, so 54.5% is an upper-bound illustration, not a tool evaluation (Appendix J). The rules do not change the cohort result above.

Scan-scope correction. Semgrep skips `dist/` and `build/` by default. Our first runs inherited that default and understated the public packs (6 of 88; cohort 7 of 28 from 5 of 19 Semgrep detections). Every figure here is from the rerun with the ignore list disabled; two more cohort findings (F-0023, F-0027) are now detected, both hard-coded keys.

6.7 Refuted priors and self-directed downgrades

Two priors about the literal rules were refuted. We expected the discarded-`verify()` rule (M10) to fire often; across 794 npm packages it gave one candidate, a JWT false positive. We expected the SM3-as-MAC rule (M8) to find misuse; its 69 hits in 37 packages were all correct HMAC-SM3. These concern the two literal rules only: the census codes M8 and M10 in other rows, reached by other routes (Tables 14, 15).

Three findings were downgraded by our own measurements. F-0011 was first written up as "decrypt with public data alone"; its pair is not self-consistent and the claim was withdrawn. F-0004 and F-0017 first rested on "V8 seeds `Math.random()` from 64 bits, so the key space is brute-forceable"; 24 independent Node processes produced 24 distinct first outputs, refuting it; the findings stood at entropy defect until §5.2's state-recovery result replaced it with a narrower, stronger claim. F-0013's key had been reported publicly before us; we claim verification, not discovery.

7. Discussion and Limitations

The taxonomy gap is structural - an argument, not a measured result. International misuse taxonomies treat the algorithm as a black box taking a key and a plaintext; in SM2 the signer's identity, the curve and the ciphertext arrangement are inputs to the operation, so retargeting a detector needs those inputs, not only new sinks. The four published tools we ran see none of them (§6.6). A limited Guomi rule set of our own does localise 48 of 88 seeded defects (§6.6, same-author bias), so we claim a gap in the taxonomies we read, not that no rule extension could express it.

A provider can be wrong in a way its own API cannot report. Bouncy Castle 1.57's JCA signatures (fixed in 1.59) are valid and self-consistent but sign SM3(M) under an empty identity, so round-trip tests pass. Self-consistency checks are blind to such defects; only a calibrated external reference catches them.

Other national suites. We did not examine GOST, ARIA, SEED or KCDSA implementations. Whether they hide security inputs behind international APIs as Guomi does is a question for their own oracles; nothing here measures it.

Most damage is still generic. Every demonstrated confidentiality break but one is a generic category (hard-coded keys, constant IVs, predictable or public values as keys); the exception is F-0025, a signature that carries the private key. M3, M4 and M5 yielded conformance defects and permitted-but-weak defaults. Novelty and severity sit in different places, and we report them separately.

What should change. For API designers, the findings suggest three rules: make the SM2 identity a required argument, make SM4 mode and SM2 arrangement required parameters of wrapper APIs, and have a JCA provider refuse an unhonoured `AlgorithmParameterSpec` instead of substitut-

ing a constant. For detector builders, the Guomi API surface needs its own sources and sinks, and some checks are cheap oracles, not rules: whether a shipped scalar `d` satisfies `[d]G = Q` separates a usable key from a decoy.

Limitations. Appendix F gives the full threat list.

- *Recall.* We lack population ground truth. Literal/data-screen disagreement is 15.8%; recall on 88 author-seeded defects is 37.5%, not corpus recall, nor does the Java figure transfer (both §6.5). The 46-encoding structural battery and 15-table negative control are battery-relative: ten further transformation classes defeat nine, with six silent negative controls (T12).
- *Impact.* Findings stop at shipped artefacts, and measured exposure is lopsided: most npm findings sit in little-downloaded packages (`U46` had few downloads in a year; none of our 794 packages depends on F-0021's library), while the two strongest findings are in Gitee applications with tens of thousands of stars each (§5.2, §5.4). The dependents we found through deps.dev are declared manifest edges, not reachability.
- *Frames.* Maven does not represent application use. Gitee covers 59 of 1415 repositories; lifting its file cap adds five candidates (three affirmatively adjudicated not defects, two structural-only), which moves the rate from 52.6% to 45.5% (§6.5). Failed npm search terms omit 168 names (Appendix E).
- *Post-hoc grading.* Rules C1-C26 were written after the inter-rater study, and C21/C23 and three L5 rows were re-graded afterwards (Appendices C.9, C.10). The census is a ledger of rules and data refined together, not a pre-specified measurement. F-0009 and F-0014 (§5.1), the membership counts (§6.3) and the detector-rule absence (§6.6) use none of its levels; §6.6's category split uses the census coding.
- *Artifact provenance.* All 124 admitted npm rows stand on registry tarballs frozen by sha256, and a records-only recheck re-downloaded them byte-identically (one package, unpublished after our analysis, was recovered from two identical archive copies). Gitee provenance is weaker: of the admitted rows, six are verified at their recorded commit and the rest stand on vendored copies, history inference or unpinned HEAD, which shows presence at check time only. The recheck re-ran no runtime oracle; a separate bounded re-execution of 27 L5 rows against their frozen tarballs recovered every victim value. Neither is independent validation. Details, including the depth of verification by level: Appendix F.
- *Adjudication.* The automated second pass was not blind (κ 0.84 before reconciliation); agreements were not re-adjudicated. Shared errors remain invisible; an independent human second rater is needed.

8. Related Work

Misuse detection and measurement. Static detection spans rule-based slicing (CryptoLint [1]), specification-driven data flow (CrySL/CogniCrypt [2]), precision-focused slicing (CryptoGuard [3]) and other languages (LICMA [5], CryptoGo [6], Gopher [7], JScamd [8]). All encode JCA, Node or Web-platform crypto: none of the 26 works read in full names an SM API or SM-specific property or uses Gitee. Benchmarks document false alarms and missed variants, including "unnatural" misuse [9-14, 17]. The closest prior work to confirmation - CRYLOGGER [15] (logged parameters against rules) and Xia et al. [16] (LLM detection with code-and-analysis validation) - confirms no static finding on the shipped artefact against a calibrated oracle with a negative control. The benchmarks CryptoAPI-Bench [17], CamBench [10] and the mutation study MASC [9] (full texts read) are Java/JCA by design, so they cannot contain Guomi cases; the taxonomy gap is a statement about what the misuse catalogues we read contain, not about what their tools could be extended to express (§7). The Chinese-language works we found on SM implementations concern library fuzzing [20] and scheme design [31]; CNKI and Wanfang article corpora were not reachable, so Chinese coverage is partial (Appendix H).

Newer detectors, side channels, ecosystem studies. Recent detectors fuse code semantics with dynamic execution traces (a multi-modal framework [36]) or fine-tune LLMs on JCA misuse (CryptoLLM [37]; [16] compares LLMs with rule-based tools). Like the tools above they target JCA-style APIs, and we did not run them on SM code. The SM literature on implementations is mostly about leakage and faults in a correct algorithm: timing, cache and EM leakage in OpenSSL's SM2 [22] and persistent faults in SM4 libraries [23]. Our oracle checks outputs and contracts and does not test timing, so side-channel exposure of the packages we studied is out of scope. Ecosystem studies measure CVE-listed flaws in cryptographic libraries [38] and how vulnerabilities propagate through npm dependencies [39, 40]. We measure unreported defects in the shipped artefacts of one algorithm family; our dependent counts are declared manifest edges, not reachability (§7).

9. Conclusion

Guomi exposes a contract gap: the identity, curve and ciphertext arrangement can disappear behind APIs designed for other algorithms. A calibrated external oracle distinguishes this gap from surface matches and self-consistent errors; the four published detector families we evaluated do not cover its semantics. Our strongest demonstrated damage nevertheless comes from generic defects, including known keys and constant IVs, not from M3, M4 or M5.

The published-artefact census admits 144 package-frame

pairs on recorded evidence, with 140 replay-supported (§5). Its npm result is concentrated: one recorded grouping contributes 54 of 124 confirmed rows, not a verified lineage. Conversion is 124/219 = 56.6%; removing those rows from the numerator gives 70/219 = 32.0%, or 70/165 = 42.4% from both sides; by finding key the grouping is one of 64 - counting scenarios, not remeasured rates. Adjudication lacks an independent human second pass, and package evidence does not establish deployment or compromise. Within those limits: expose the algorithm's security inputs, and test provider behaviour rather than assume a successful call honours it.

References

[1] Egele, Brumley, Fratantonio, Kruegel. An Empirical Study of Cryptographic Misuse in Android Applications. ACM CCS 2013. doi:10.1145/2508859.2516693

[2] Krüger, Späth, Ali, Bodden, Mezini. CrySL: An Extensible Approach to Validating the Correct Usage of Cryptographic APIs. IEEE TSE 2021. doi:10.1109/tse.2019.2948910

[3] Rahaman, Xiao, Afrose, Shaon, Tian, Frantz, Kantarcioglu, Yao. CryptoGuard: High Precision Detection of Cryptographic Vulnerabilities in Massive-sized Java Projects. ACM CCS 2019. doi:10.1145/3319535.3345659

[4] Hazhirpasand, Ghafari, Nierstrasz. Java Cryptography Uses in the Wild. ESEM 2020. doi:10.1145/3382494.3422166

[5] Wickert, Baumgärtner, Breitfelder, Mezini. Python Crypto Misuses in the Wild. ESEM 2021. doi:10.1145/3475716.3484195

[6] Li, Jia, Liu, Zheng, Ma, Lin. CryptoGo: Automatic Detection of Go Cryptographic API Misuses. ACSAC 2022. doi:10.1145/3564625.3567989

[7] Zhang, Li, Lin, Li, Bai, Jia, Wu. Gopher: High-Precision and Deep-Dive Detection of Cryptographic API Misuse in the Go Ecosystem. ACM CCS 2024. doi:10.1145/3658644.3690276

[8] Jia, Xu, Ma, Niu, Liu, Zha, Lin. An Automated Static Taint Analysis Framework for Detecting Cryptographic API Misuses in JavaScript (JScamd). USENIX Security 2026.

[9] Ami, Cooper, Kafle, Moran, Poshyvanyk, Nadkarni. Why Crypto-detectors Fail: A Systematic Evaluation of Cryptographic Misuse Detection Techniques (MASC). IEEE S&P 2022. doi:10.1109/sp46214.2022.9833582

[10] Schlichtig et al. CamBench — Cryptographic API Misuse Detection Tool Benchmark Suite. MSR 2022 (registered report). arXiv:2204.06447

[11] Zhang, Kabir, Xiao, Yao, Meng. Automatic Detection of Java Cryptographic API Misuses: Are We There Yet? IEEE TSE 2023. doi:10.1109/tse.2022.3150302

[12] Chen, Liu, Wu, Le, Chau. Towards Precise Reporting of Cryptographic Misuses. NDSS 2024. doi:10.14722/ndss.2024.241032

[13] Wickert, Baumgärtner, Schlichtig, Narasimhan, Mezini. To Fix or Not to Fix: A Critical Study of Crypto-misuses in the Wild. IEEE TrustCom 2022. doi:10.1109/trustcom56396.2022.00051

[14] Olaiya, Nadkarni. From Base Cases to Backdoors: An Empirical Study of Unnatural Crypto-API Misuse. arXiv:2510.13102, 2025.

[15] Piccolboni, Di Guglielmo, Carloni, Sethumadhavan. CRYLOGGER: Detecting Crypto Misuses Dynamically. IEEE S&P 2021. doi:10.1109/sp40001.2021.00010

[16] Xia, Xie, Liu, Lu, Liu, Wang, Ji. Beyond Static Pattern Matching? Rethinking Automatic Cryptographic API Misuse Detection in the Era of LLMs. ISSTA 2025. doi:10.1145/3728875

[17] Afrose, Rahaman, Yao. CryptoAPI-Bench: A Comprehensive Benchmark on Java Cryptographic API Misuses. IEEE SecDev 2019. doi:10.1109/secdev.2019.00017

[18] Vranken et al. Cryptofuzz: differential fuzzing of cryptographic libraries. https://github.com/MozillaSecurity/cryptofuzz

[19] Zhou, Ma, Chen, Ren, Jiang. Vulnerability Detection of Cryptographic Algorithm Implementation via Semantic-aware Fuzzing (CLFuzz). ACM TOSEM 2023. doi:10.1145/3628160

[20] Ma, Zhou, Chen, Yan, Jiang, Sun. 密码算法库模糊测试技术研究综述 (A survey of fuzzing for cryptographic algorithm libraries). Journal of Cryptologic Research 11(3):504–520, 2024. doi:10.13868/j.cnki.jcr.000693

[21] Project Wycheproof (C2SP). https://github.com/C2SP/wycheproof

[22] Tuveri, ul Hassan, Pereida García, Brumley. Side-Channel Analysis of SM2: A Late-Stage Featurization Case Study. ACSAC 2018. doi:10.1145/3274694.3274725

[23] Guo, Ke, Wang, Zheng. Persistent Fault Analysis Against SM4 Implementations in Libraries Crypto++ and GMSSL. IEEE Access 2021. doi:10.1109/access.2021.3074708

[24] NVD. CVE-2026-23965: sm-crypto SM2 signature verification forgery, fixed in 0.4.0. https://nvd.nist.gov/vuln/detail/CVE-2026-23965

[25] GB/T 32918.4-2016 and GB/T 32918.5-2017. Information security technology — Public key cryptographic algorithm SM2 based on elliptic curves, parts 4 and 5.

[26] GB/T 32905-2016. Information security techniques — SM3 cryptographic hash algorithm.

[27] GB/T 32907-2016. Information security technology — SM4 block cipher algorithm.

[28] GM/T 0009-2023. SM2 cryptography algorithm application specification.

[29] GM/T 0003.4-2012 and GM/T 0010-2023 (SM2-related cryptography industry standards; scanned documents without a text layer; the relied-upon GM/T 0003.4-2012

clauses read from the official page images; GM/T 0010-2023 held as an OCR transcript).

[30] Oracle. Java SE 17 API: `javax.crypto.CipherSpi` (`engineInit`, `engineGetIV`).

[31] Yang. 面向PLC的轻量级间歇性匿名消息认证方案 (A lightweight intermittent anonymous message authentication scheme for PLCs). M.Sc. thesis, Southwest University, 2024.

[32] Semgrep OSS 1.182.1 and its public rule packs `p/java`, `p/javascript`, `p/secrets`, `p/gitleaks`.

[33] SpotBugs 4.9.8 with the FindSecBugs 1.14.0 plugin.

[34] bc-java pull request #218, "Fix an incorrect of SM3WithSM2 signature algorithm" (2017-08-31), https://github.com/bcgit/bc-java/pull/218; fix commit b071aa17 (2017-09-01); Bouncy Castle 1.59 release notes (2017-12-28).

[35] NVD. CVE-2026-73567: sm-crypto SM2 key generation and signing use insufficiently random values, fixed in 0.5.0. https://nvd.nist.gov/vuln/detail/CVE-2026-73567; GitHub Advisory GHSA-vh45-f885-3848 (repository advisory text updated 2026-10-27); fix commit https://github.com/JuneAndGreen/sm-crypto/commit/1f9bd7bd160c24efd9c26c8f7fda997c68c823d0

[36] Zhou, Jia, Niu, Zha. Fusing Code Semantics and Dynamic Execution Traces: A Multi-Modal Framework for Detecting Cryptographic API Misuse. ICICS 2026. doi:10.1007/978-981-92-6843-6_21

[37] Baek, Lee, Kim. CryptoLLM: Harnessing the Power of LLMs to Detect Cryptographic API Misuse. Springer LNCS 14982 (ESORICS series), 2024. doi:10.1007/978-3-031-70879-4_18

[38] Blessing, Specter, Weitzner. Cryptography in the Wild: An Empirical Analysis of Vulnerabilities in Cryptographic Libraries. ACM ASIA CCS 2024. doi:10.1145/3634737.3657012

[39] Zimmermann, Staicu, Tenny, Pradel. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. USENIX Security 2019. arXiv:1902.09217

[40] Zerouali, Mens, Decan, De Roover. On the impact of security vulnerabilities in the npm and RubyGems dependency networks. Empirical Software Engineering 27, 2022. doi:10.1007/s10664-022-10154-1

Ethical Considerations

Stakeholders and data. The stakeholders are the maintainers of the affected packages and repositories, their users, and the registries. Every artefact we studied is a published package or a public repository, and every test ran locally against a copy of the artefact: no deployed system was probed, no traffic was intercepted, and no account or credential of any third party was used. The study involves no human subjects. The disclosure-channel measurement (Appendix G) reads public registry metadata only and contacts nobody, so it cannot create the contact it measures.

Responsible disclosure. Findings that are vulnerabilities were reported privately to the maintainer, by email to the address the package or repository publishes, with a request for a 90-day window (GitHub private vulnerability reporting was unavailable for all of them). Letters went out between 2026-10-11 and 2026-10-27, so most windows close in January 2027, around the submission date. Until a window closes, this preprint pseudonymises the affected artefacts (U1-U61) and withholds identifying versions and code fingerprints; names return at camera-ready when the window has ended, a fix or advisory is public, or the maintainer agrees. The letters ask for 90 days before any public write-up that names the package. For maintainers who reply slowly or not at all, the window ends 90 days after the first notice (2027-01-22 to 2027-01-25 for the letters of 2026-10-24 and 2026-10-27); a date agreed with a maintainer replaces it, and a maintainer who replies with real progress may get more time. About two weeks before the end we send a final notice on the original channel. A release then names the package and gives the bug class, the affected versions, how the behaviour was reproduced and fix advice, as a public advisory with a CVE request where possible; ready-to-use exploit code is not released. Each letter carries its evidence (artefact coordinates and hash, the measured behaviour, a negative control, the standard or contract text, the affected versions), says what we do not claim, and asks questions instead of giving a verdict.

Status on 2026-11-01. One maintainer confirmed a report and published a security advisory on 2026-10-28; a CVE was requested the same day and had not been assigned at the last check (2026-11-01). Another maintainer proposed a public date of 2026-11-11, pending his confirmation; until he confirms, the 90-day date in the letters (2027-01-25 for those sent on 2026-10-27) stands. Two final bounces arrived on 2026-10-29, for one address whose server dropped the connection at greeting and one whose tenant rejects unauthenticated external mail. The other recipients of those two letters did not bounce, and each letter reached the package's repository owner or other registered maintainers; no verified further private address exists for the two bounced ones, and a public issue would disclose more than the letters did, so no resend was made. The mailbox check of 2026-11-01 found no further maintainer replies and no new bounces.

Every census key except F-0013 (whose key a third party had already published) is covered by at least one letter, but sent is not delivered and delivered is not fixed. Delivery is incomplete for some: the only registry address of two packages does not exist, some letters had one or more addresses bounce, one letter is held after two failed submissions, and five older letters have unverified delivery. One maintainer replied that the package was unpublished and unmaintained. Table 20 (Appendix J) lists the maintainer-side responses so far; it is the only evidence in this paper not produced by our own adjudication. Four maintainers have replied so far; we report no reply rate, because the letters are between 2 and 18 days old and delivery is unverified for some. The letter log, with dates and failures, is in Appendix G. F-0009 is not a vulnerability, so it was posted as a public, non-security question to the library's maintainers; a maintainer replied that it is the known issue #218, fixed in 1.59 [34]. Findings graded as permitted behaviour (F-0002, F-0015, F-0008's 1.59+ half) or as divergence without a security property (F-0001) are reported as such and not disclosed as vulnerabilities.

Harms of publication and mitigations. Several findings concern keys that protect live projects. This paper therefore prints no private key, SM4 key or IV belonging to a specific project, except values that are already public as standard example constants (such as the GB/T 32907 example key that F-0021 and F-0023 fall back to). Where a third party published a finding first (F-0013, and an earlier plaintext version of the F-0024 server key), we claim verification and not discovery. Severity vocabulary we cannot support (CVSS scores, "affects many systems") is avoided, because downstream use was measured and found zero or unmeasurable.

Dual use. The oracle and the state-recovery solver could help an attacker as much as an auditor. The solver only applies under a stated precondition (observation of `Math.random()` outputs in the same isolate), and the oracle reproduces public standards. No release is pre-approved: each tool is released, if at all, only after a per-target review of its terms, the current public status of each finding it touches, and the residual risk, not on the calendar alone.

Open Science

Archive and access. Our private research archive contains the reference implementations (`harness/refimpl/`), screens and adjudication scripts (`harness/screen/`), recorded measurements (`analysis/`), finding records (`findings/`), the seeded benchmark (`harness/bench/`), the earlier automated second-coder package (`irr/`), and tests (`tests/`). A frozen calibration-only snapshot is available at https://anonymous.4open.science/r/reference-calibration-FDD5/. It supplies reference code, published example-vector checks, source notices and file hashes, not the empirical archive. Download the ZIP and run `python run_checks.py` with Python 3 and no dependencies: 15 existing assertion functions pass. The included SM4 million-iteration check is opt-in and was not rerun. The mirror is accessible to anyone holding the link, not PC-authenticated; it is frozen without auto-update and expires by removal on 2027-10-27.

Omissions and their cost. The archive interweaves project-specific probes, unanswered or unsent coordinated-disclosure evidence, and files reproducing live projects' key material. These are withheld from a bearer-link mirror to avoid an uncontrolled disclosure audience. Copyrighted full standards and the cited thesis are not redistributed; clauses are identified by page. Third-party corpora also have redistribution constraints. The supplied calibration-only subset does not support the empirical contributions and is not presented as the complete artifact. Reviewers cannot independently rerun the census adjudications, the 140/144 replay-support result, the lineage analysis, or the detector comparison from the supplied subset. This limits independent assessment of those claims.

Replay package and controlled access. A second, redacted package replays the seeded-benchmark and rule results (§6.5-6.6, Appendix J): reference implementations, the 88-defect benchmark and oracle, the Guomi Semgrep rules and the recorded outputs, with hashes for every file. Secrets, finding records, letters, recipient data, the cohort and every package still under a coordination window are removed, and third-party tarballs are refetched from npm by pinned version. It is available, frozen without auto-update, at https://anonymous.4open.science/r/replay-review-snapshot-20261029-2E52/ (53 files, removed on 2027-10-27; the link is accessible to anyone holding it). For the withheld evidence we offer reviewers controlled access: read-only, through a channel agreed with the chairs, with undisclosed package names and maintainer correspondence visible only after the relevant coordination window ends or the maintainer agrees. **Controlled check of the undisclosed census.** During review, a chair can ask us for the redacted adjudication record of any undisclosed finding key by its U-number, up to 10 keys: the artefact binding (a hash, or a snapshot or commit where no archive exists), the probe, its negative control and the oracle output where recorded, and the category claim, with package names and maintainer contact removed. Recorded gaps, such as a partial oracle run or no separate negative control, are stated in the record. We return them through the chairs within three days. Ten is what one author can prepare in that time without changing the records; it is a spot check of whether adjudications match their evidence, not a replication of the census.

Any later sharing requires an actual anonymous access route compatible with the disclosure and license restrictions. A coordination window ends 90 days after the first notice for a silent maintainer, or on a date the maintainer agreed; the authors then review each key before anything is released.

Private reproduction scope. In the complete private checkout, `python -m harness.reproduce` is the reproduction entry point: it checks the reference vectors (including SM4's million-iteration vector), detector controls, file scope and frame membership, measurement files, funnel counts, paper consistency and tests. This is not a claim that readers can run that command from materials supplied with this submission. It does not re-run Appendix J's static-tool pilot (its original driver scripts are absent from this checkpoint); it cross-validates the preserved scoring record against the preserved raw tool outputs (R-0039). Requirements are Python 3.12, JDK 17 and Node 22 or 24; missing corpus inputs are reported as failures, except two that are not shipped by design (the licensed standards PDFs and the extracted-tarball working directory), which are noted. The v2 coding-time probes were not preserved; the available per-row attempt and replay records are under `analysis/v2_adjudication/oracle/`. No independent human v2 review has been performed. Reviewers decide whether these omissions are justified.

Appendix A. The Platform Verdicts in Detail

A.1 F-0009 at source level

`org.bouncycastle.jcajce.provider.asymmetric.util.DSABase.engineSign()` in bcprov 1.57:

```
byte[] d = digest.doFinal();                          // = SM3(M)
BigInteger[] sig = signer.generateSignature(d);    // passes the
↪ digest
```

while `org.bouncycastle.crypto.signers.SM2Signer.generateSignature(byte[] message)` in the same version expects the message and hashes it:

```
digest.update(z, 0, z.length);
digest.update(message, 0, message.length);         // message, not
↪ a digest
```

and, with no `ParametersWithID`, uses an empty identity. Together: `sign(SM3(M), ID = "").DSABase` is byte-identical across 1.57, 1.66 and 1.78; what changes in 1.59 is that `GMSignatureSpi` stops extending it.

The eight-path matrix (Table A1) verifies each signature against the message `M`.

Table A1. Rows are signers, columns are verifiers; `OK` = accepted.

signer	bc157-JCA	bc166-JCA	bc178-JCA	ref(empty)	ref(default)	ref(real)
bc157-JCA	OK	no	no	no	no	no
bc157-raw	no	no	no	OK	no	no
bc166-JCA	no	OK	OK	no	OK	no
bc166-raw	no	OK	OK	no	OK	no
bc178-JCA	no	OK	OK	no	OK	no
bc178-raw	no	OK	OK	no	OK	no
ref-default	no	OK	OK	no	OK	no
ref-empty	no	no	no	OK	no	no

The matrix also cross-validates our reference: its default-identity behaviour matches 1.66/1.78, its empty-identity behaviour the 1.57 low-level path. That change is commit b071aa17, Bouncy Castle's fix for bc-java #218, which reported both the repeated hashing and the missing default identity on 31 August 2017 [34]. The version matrix is produced by a reflective probe that loads each released provider jar (checked against Maven Central SHA-1 and SHA-256) and exercises both `SM3withSM2` and the low-level `SM2Signer`.

A.2 What would make F-0009 wrong

The finding would be wrong if (1) our reference's `(r,s)` order or endianness were wrong, but it reproduces the standard's published `(r,s)` byte-for-byte and swapping or byte-reversing fails (§3.1); (2) our probe entered the wrong code path, but the source localisation predicts the observed `e`, and the low-level path in the same jar is conformant; or (3) the difference were an encoding artefact, but the jar hash is recorded and source-compiled and published-artefact runs agree. A fourth possibility, that the 1.57 behaviour was an intended digest-then-sign contract, is settled: Bouncy Castle treated it as a defect in 2017 and fixed it in 1.59 [34]. The test that pins the mechanism is positive, not negative: 1.57 and 1.58 JCA signatures verify 32/32 under `(SM3(M), ID = "")` with two independent verifiers and 0/32 under every reading that uses `M`.

A.3 The JCA contract F-0014 violates

From the JDK 17 `javax.crypto.CipherSpi` documentation of `engineInit` with an `AlgorithmParameterSpec` [30]:

> If this cipher requires any algorithm parameters and params is null, the underlying cipher implementation is supposed to generate the required parameters itself (using provider-specific default or random values) if it is being initialized for encryption or key wrapping… The generated parameters can be retrieved using engineGetParameters or engineGetIV (if the parameter is an IV).

and of `engineGetIV`: "Returns: the initialization vector in a new buffer, or null if the underlying algorithm does not use an IV, or if the IV has not yet been set." The relevant lines of the SM4 cipher class of the provider in `U44` are:

```
14      private String mode = "CTR";                          //
↪  default mode is CTR
56      protected byte[] engineGetIV() { return new byte[0]; }
↪  // an IV IS used
61      protected AlgorithmParameters engineGetParameters() {
↪  return null; }
78      if (spec instanceof SM4AlgorithmParameterSpec) {   // its
↪  own type only
80          this.iv = spec.getIv();
81      }
103     if (iv == null) { iv = "1234567812345678".getBytes(); }
↪  // engineDoFinal
```

Downstream of the artefact was measured with Maven reverse-class lookup (with a control: the same lookup for Bouncy Castle's `SM2Signer` returns 21,069 artifacts, and an earlier run against a non-existent package returned zero even for the artifact's own class, which is how that bug was caught), with authenticated GitHub code search at three granularities, and with Gitee repository search. The first two return zero; the third is unmeasurable, not zero, because Gitee has no code search.

Table 11. A literal-presence screen on a known-positive corpus: the SM4-GCM implementation is present in every published output and the literal screen reports it in none of the CJS outputs.

literal searched for	present in ESM	present in CJS
`SM4-GCM`	yes	**no**
`U33`	yes	**no**
`sm3` / `sm2`	yes	**no**
SM4 S-box first row, hex	**no**	**no**
SM4 key schedule, first word, hex	**no**	**no**

Appendix B. Screen Defects and How They Were Found

B.1 A known-positive control for minification

`livekit-client-sm4@2.22.4` publishes a UMD bundle, an ESM bundle and two worker bundles (e2ee and fm), each reachable through both the `require` and `import` conditions. Reconstructing its TypeScript from the shipped sourcemaps (517 source entries with content across 148 maps, 171 unique files) confirms SM2, SM3 and an SM4-GCM wrapper over `U33`. Table 11 shows what a literal-presence screen finds in it.

The last two rows matter: the signature encodings are absent from the ESM outputs too, where the feature is unambiguously present. Any conclusion of the form "this package does not do X", reached by not finding a string, is unsupported. The structural detector, matching SM4's S-box, SM3's initial value and SM2's curve order whitespace-insensitively in several encodings, recovers all three primitives in all six published outputs of the package. Its first version matched any run of consecutive `0x` words without naming the standard's bytes, and so fired on SHA-256's round constants and BLAKE2's IV; two of the eleven initial sample hits were that error, which is why every structural hit in the sensitivity sample was confirmed by reading the constant in context.

B.2 Encodings, in both directions

The structural detector was built on six JavaScript bundles while the population admits `.java`, `.go`, `.py`, `.cs`, `.php` and `.vue`. A positive control writes the SM4 S-box, the SM3 initial value and the SM2 parameters in 46 spellings across JavaScript, Java, Go, Python, C#, PHP, Vue single-file components and bundler output; the first run found seven the detector could not see (a space-separated hex string in Java and C#, a Python byte literal, `0x`-grouped hex with underscores, binary literals, and `new BigInteger("...")` without a radix), all since added. A negative control runs 15 tables that a loose encoding could confuse with Guomi data (SHA-256 constants, the AES S-box, CRC-32, synthetic ascending tables, 40-digit decimal constants); 15 of 15 stay silent. Both controls are pinned in tests that take their cases from the modules that own them.

B.3 The screen was blind on this study's own findings

When the re-screen was extended to all frames we first checked that the confirmed findings were still inside the candidate set, since a larger N3 is a correction only if it is a superset. F-0013 was not. Its defect is a single assignment to `PRIVATE_KEY`, and `M1_hardcoded_key_or_iv` enumerated key-material identifiers as camelCase literals. `re.I` makes a pattern case-insensitive; it does not remove an underscore. So `PRIVATE_KEY`, `SM4_KEY` and `sm4_key` did not match while `privateKey` did. F-0012's key material, in names such as `RESPONSE_SM2_PRI_KEY`, was never matched either; its file was a candidate only because a bare `iv` alternative matched the end of `DB_SM4_CBC_IV`. A rule written from one language's naming habits is systematically blind in another, and here the bias pointed at the Java and C frames.

The first repair was itself defective, and a hit rate caught it. The pattern was assembled as `_KEYISH + r"|[A-Za-z0-9_]*\bsecret\b" + r"\s*[:=]=?\s*..."`; because `|` has the lowest precedence, the concatenation re-grouped it and the "must be assigned from a non-CSPRNG" tail applied only to the second branch. The rule then matched any identifier ending in `key` and named 56 of the 59 Gitee repositories. Attributing every new hit to a pattern and printing the matched line showed a comment, an import and a function parameter. A test now asserts behaviour, not regex text: any fragment with a top-level `|` must be parenthesised before concatenation. The same widening made the screen non-terminating on a 35 MB bundle containing a 25,484-character alphanumeric run, because an unbounded `[A-Za-z0-9_]*` before a required token is quadratic in such a run; the leading wildcard was removed, and the screen now runs at about 3 MB/s.

The surviving rule matches identifiers structurally: a Guomi or cryptographic algorithm token anywhere in the name, a key-material role token, or a bare material word. `PUBLIC_KEY` alone stays out; `SM2_PUB_KEY` is admitted as a candidate; `apiKey` and `appSecret` remain outside the category by design. (In the code, the pattern for the M10 category of Table 12 is named `M7_verify_result_ignored`.)

B.4 Scope and verdict hygiene

The shared file scope omitted five extensions that its own frame screens covered (`.kt`, `.scala`, `.groovy`, `.rs`, `.rb`); a missing extension is the claim "this language has no SM usage" made by omission. The relation is now asserted in code, not listed by hand: the shared scope must be a superset of every frame screen's extension list, checked on every execution and in the test suite. Screen verdicts are stored under a digest of the pattern set, after shards written by an older pattern set were found on disk and a merge produced a count from 793 stale verdicts and one fresh one.

B.5 The nine precision corrections

Row counts are as recorded in the project ledger at the time of each correction. Row 2's after-count and row 3's before-count cannot be reconstructed from any committed screen snapshot: the nearest committed logs show 68 hits in 8 packages and 35 hits in 12 packages. The table cites the ledger as it stands.

Appendix C. Application-Layer Evidence

Auxiliary reference code outside the adjudication-used oracle paths (the SM4-GCM anchor code) carries its own stated coverage limits and supports no verdict.

C.1 Deciding an arrangement that length cannot decide

`U45` passes `cipherMode = 0` to `sm-crypto`. `C1‖C2‖C3` and `C1‖C3‖C2` have the same length, so no field-size argument distinguishes them. What settles it is integrity: C3 is an SM3

Table 12. The nine precision corrections, with before and after counts (hits / packages).

#	pattern	before	after	cause
1	M6 bad nonce	~1000 hits, nearly every package	14 / 8	matched "the package calls `Math.random` anywhere"
2	M4 C1C2C3	556 / 79	77 / 8	matched `cipherMode = 0`, i.e. every configuration object
3	M1 hard-coded key	33 / 11	6 / 3	matched any long hex literal, incl. a Kugou music API secret
4	M8 SM3-as-MAC	69 / 37	**0**	every hit was `if (key.length > blockLen) key = sm3(key)` — the RFC 2104 HMAC key-shortening step
5	M11 public-value-as-key	—	narrowed	`publicKey.substring(0,64)` splitting into X/Y is *correct* SM2 usage
6	screening scope	—	narrowed	`.d.ts` and `test/` excluded — docs and fixtures
7	M10 ignored verification	10 / 3	7 / 2	matched the last line of a chain whose result is *returned* (alipay-sdk)
8	M6 signer verbs	60 / 27	35 / 17	no word boundary, so `Object.ass`**`ign(`** matched `sign(`
9	M6 nonce branch	60 / 27	35 / 17	a *legitimate* `timestamp=Date.now()` 40 chars from `nonce=` matched

Table 13. Adjudicating `sm-crypto`'s `cipherMode` with our own SM2 implementation.

`cipherMode`	our reference `decrypt()`	arrangement
0	`ValueError: C3 mismatch`	not C1‖C3‖C2
1	recovers the plaintext exactly	**C1‖C3‖C2**, conformant

digest over the shared point and the plaintext, so only the correct placement verifies. Our GB/T 32918.4-2016 implementation decrypts both ciphertexts that the published `sm-crypto` 0.1.4 produces and rejects a C3 mismatch (Table 13).

The library's default (`1`) is conformant; the non-conformant choice is the application's, so upgrading the dependency would not remove it. The probe of F-0017 itself records 16 `Math.random` calls per encryption, a 16-character alphanumeric session key, and recovery of the payload with that key alone and no SM2 private key. The same module uses SM4-ECB on its SM4 and MIX paths, recorded but not separately verified.

C.2 F-0021

The exported singleton is

```
Vt = {   // 100 to 100000
  _BIG: Lt(globalThis.Math.round(globalThis.Math.random() * 99900
  ↪  + 100)),
  _KEY: "0123456789abcdeffedcba9876543210",
  getDH(g, p, server_number) { ... }
}
```

and `getDH(7, 2147483647, 5)` returns `key` = the constant, `bigKey = 436690296` (= 5^97403 mod 2147483647) and `client_number = 743912402` (= 7^97403 mod 2147483647). `decrypt(encrypt("AAAAAAAAAAAAAAAA", 7, 2147483647, 5).result, getDH(...).key)` returns the plaintext. Its download series is 365 days at a mean of 1.92 per day, peak 70, with 163 days at zero. That series was only quotable after a control: npm's downloads endpoint called with a colon instead of a slash between period and package silently ignores the package and returns the same dataset for every name (960,444,269,343 for both `react` and a package with 699 downloads a year); the slash form sums exactly to the point aggregate, with `react` as the control.

C.3 F-0073 (the retired v1 F-0005 record), F-0006 and F-0010

U5 declares an enum selecting between the two arrangements and its encrypt and decrypt call sites both select the non-standard order; in all eleven published versions, all 22 call sites do. Of the 147 packages in its scope, one same-vendor application pins the defective version, whose call sites also select it; a second package pins an unpublished version and is not counted. The deps.dev index (queried 2026-10-26) adds one further same-vendor direct dependent on the defective version (a release candidate); the latest published version lists none. A static read of the published tarballs (2026-10-26) confirms reachability: all three release-candidate bundles vendor the plugin's SM2 wrapper, ship constants byte-identical to its built-in public and private keys, keep the no-appId fallback to the built-in private key, and hard-code the non-standard order at their own call sites (two defaults enabled); the socket line publishes release candidates only, pins the prior library version from its first candidate on, and carries the same constant and fallback through its latest candidate. As for the key itself, the live-service condition is untested. (recorded reachability note and the two tarballs in the private archive.) For F-0006, `"A".repeat(64)` encrypts to four identical blocks, and a structured plaintext yields an identical pair of blocks;

GM/T 0009-2023 §7.4 ends with the symmetric algorithm identifier `SGD_SM4_ECB` for SM2 key wrapping, which is why the finding is conditional. For F-0010, `sm4_cbc_encrypt` and `sm4_cbc_decrypt` both use one hard-coded IV, and our own SM4 reproduces both leak properties and their disappearance under a different IV.

C.4 Shipped keys

For every shipped SM2 private key we compute `[d]G` with the reference implementation and compare it with the shipped public key. It matches for F-0012 (both pairs), F-0013 and F-0024, and does not for F-0022 and F-0011. F-0011's default path therefore fails silently: two identical empty strings, no exception, while the same code with a consistent pair round-trips, which places the defect in the key material. F-0016's scalar is valid (255 bits, on the curve) and used by two of its TLS server class's three constructors; the package ships 21 TLS classes and no caller of those constructors was found. F-0023's `U52` fallback key is used by the request interceptor on every outbound body - the probe recovers the plaintext from wire data under the default configuration - while how many callers omit `sm4Key` is not measured. In `U53` the same constant fallback exists in the source but is unreachable in the module's normal flow: its setup hook generates and installs a key and IV in the same context update that enables encryption, so the re-adjudication scores its fallback M1:N and the package's defect stands on M9/M12; a correction narrowing the sent letter was sent on 2026-10-26 (Appendix G). F-0003's probe returns the plaintext of a test message from the public key alone.

C.5 Where M1 does not apply

A Chinese-language thesis on PLC message authentication [31] advances hard-coding secret material, including all SM4 round keys, as a deliberate countermeasure, on the premise that PLC program code is unreadable once the controller is in run mode. In published npm tarballs, Maven artifacts and public Gitee repositories the code is readable by anyone, so the premise fails and our M1 findings stand; we do not evaluate the thesis's own threat model. We state the premise because a rule that flags hard-coding without it turns a domain difference into a universal defect.

C.6 F-0019

`cipherEncrypt(key, plaintext, null)` throws, while `cipherDecrypt(key, ciphertext, null)` substitutes the ASCII bytes of `"1234567812345678"` and succeeds. Encrypting with an explicit IV and decrypting with none returns wrong plaintext without an exception, because CTR cannot detect a wrong keystream. No caller pairs the calls this way and no confidentiality consequence follows, so it is a correctness defect.

C.7 The v2 defect census

Tables 14 and 15 give the complete census: the 64 finding keys over the 144 confirmed package-frame pairs (147 adjudicated; §5), with per-key frames, package counts, categories and standing. Categories are adjudicated assignments, independent of level: a listed M4 or M14 claim may stand at L2 (conformance) after the 2026-10-26 re-grade (Appendix C.9). For ten of the fourteen caveat rows (C.11) the recorded re-run did not re-check every claimed category (the per-row oracle records list the coverage), so category frequencies read as adjudicated, not uniformly re-verified. The remaining four caveat rows (`U51`, `U60`, U34, U35) are module-syntax shims: C24 was not triggered because the documented bundler/ESM path loads. Three keys' category assignments were corrected on 2026-10-27 against the precise taxonomy definitions (a category-review note; LLM/agent records adjudication, human re-validation pending): F-0057 (M1 removed - computed-from-public key is M11, not a hard-coded literal), F-0060 (M14 removed - the counter-carry bug is M9, not contract nonconformance) and one F-0032 member (M3, M6, M13 removed as unsupported-by-record on 2026-10-27; on 2026-10-28 the row's byte-identical artifact was recovered (§7, artifact provenance) and M6 and M13 were restored on static evidence against the exact taxonomy criterion - M13's bundled-code criterion, not mere vendoring presence: the shipped dist is a fully self-contained bundle with the vulnerable jsbn pattern inlined and reachable, immune to the declared dependency's upgrades - while M3 stays removed, so the member's effective categories are M6+M12+M13+M14 as of this revision; the key's category union is unchanged throughout).

The sentinel per-package write-ups were added on 2026-10-28 with outcomes as originally recorded. IRR-02/IRR-33 is one merged record covering two packages.

C.8 The F-0032 grouping

One finding key, F-0032, contributes 54 of the 147 adjudicated rows, all npm, all within the 144 confirmed pairs (§5). The 54 rows are a recorded adjudication grouping - every member carries M12 (weak entropy) - not a verified single-origin family; the decompositions below show it mixing several implementations and constructs. Because 54 rows is more than a third of the census, we decompose the census four ways and choose no single counting unit.

By category. No reported category is grouping-only: each of M1-M14 is also observed in non-grouping rows (observed, not necessarily oracle-executed: the census's two non-executed rows are non-grouping, §5). Four categories are grouping-majority by row count - M6 (41 of 52 rows), M7 (17/23), M12

Table 14. The v2 defect census, npm keys (46 of 64; Gitee and cross-frame keys continue in Table 15): 64 finding keys over the 144 confirmed package-frame pairs (147 adjudicated; §5). Categories M1-M14 as defined in Table 1. Standing is the highest level among the key's packages: L3 misuse in shipped code, L4 security property violated on the shipped artefact, L5 with a demonstrated consequence (§3.3). F-0038's and F-0040's standings are the original recorded verdicts: this review adds no new verification evidence for their L5, which the 2026-10-26 record-based re-review affirmed (Appendix C.10); F-0040's M3 basis was re-checked statically against the shipped artefact. Two sentinel keys group L3 packages (L3-UNRECORDED-NPM: 15 packages; L3-UNRECORDED-GITEE: 2) whose per-package write-ups and per-package rows are in the research archive. Their evidence records exist independently of the write-ups; one M10 member's record is representative: U39, where a C2-tampered SM2 ciphertext decrypts to the correspondingly altered plaintext and a tampered C3 integrity value is accepted without error. IRR-02/IRR-33 is one merged record covering two packages.

ID	frame	packages	cat.	standing
F-0003	npm	1	M2+M4+M11	L5
F-0004	npm	1	M2+M12+M13	L4
F-0006	npm	1	M2+M3+M14	L5
F-0010	npm	1	M1+M6+M9+M12+M13	L5
F-0011	npm	1	M1	L4
F-0017	npm	1	M2+M4+M6+M12+M13	L5
F-0021	npm	1	M1+M12+M13+M14	L5
F-0022	npm	3	M1	L3
F-0023	npm	2	M1+M2+M9+M12	L5
F-0025	npm	1	M11+M14	L5
F-0026	npm	1	M12	L4
F-0027	npm	1	M1+M8+M12	L4
F-0028	npm	6	M1+M2+M9+M12+M13+M14	L5
F-0029	npm	1	M1	L3
F-0032	npm	54	M2+M3+M4+M5+M6+M7+M9+M10+M12+M13+M14	L5
F-0037	npm	1	M9+M11+M12+M14	L5
F-0038	npm	1	M6	L5
F-0039	npm	1	M6+M7+M12+M14	L5
F-0040	npm	1	M3+M6+M14	L5
F-0041	npm	1	M9+M14	L5
F-0042	npm	1	M2+M11+M12	L5
F-0052	npm	1	M14	L4
F-0053	npm	1	M3+M14	L4
F-0054	npm	1	M2+M4+M8+M9+M14	L3
F-0055	npm	1	M8	L4
F-0056	npm	1	M14	L4
F-0057	npm	1	M2+M11	L4
F-0059	npm	1	M2	L4
F-0060	npm	1	M9	L4
F-0061	npm	1	M2	L4
F-0062	npm	1	M2	L4
F-0064	npm	1	M2	L4
F-0065	npm	1	M14	L4
F-0066	npm	1	M2	L4
F-0067	npm	1	M9+M14	L4
F-0068	npm	1	M9+M14	L4
F-0069	npm	1	M4+M6	L3
F-0070	npm	1	M2	L4
F-0073	npm	1	M1+M4	L5
IRR-02/IRR-33	npm	2	M1+M2+M14	L5
IRR-18	npm	1	M1+M2+M14	L3
IRR-32	npm	1	M2+M6+M9+M12+M13	L5
IRR-34	npm	1	M1+M2	L4
IRR-46	npm	1	M9+M12	L4
IRR-55	npm	1	M9	L3
L3-UNRECORDED-NPM	npm	15	M1+M2+M3+M7+M8+M9+M10+M12+M14	L3

Table 15. The v2 defect census, continued: the 18 Gitee and cross-frame keys. Definitions and sentinel notes as in Table 14. Gitee provenance is not uniform over the 20 admitted rows behind these keys: six re-verified at their recorded SHA (one more corroborated at an unchanged HEAD); thirteen rest on vendored copies or unpinned HEAD-consistency at clone time (Appendix I).

ID	frame	packages	cat.	standing
F-0007	gitee	1	M4+M10+M14	L5
F-0012	gitee+npm	4	M1+M9+M12+M13	L5
F-0013	gitee	1	M1	L5
F-0014	gitee	1	M1+M9+M14	L5
F-0024	gitee	1	M1	L5
F-0033	gitee	1	M1	L5
F-0034	gitee	1	M5+M6+M7+M10+M13+M14	L5
F-0035	gitee	1	M1+M4+M10	L5
F-0036	gitee	1	M5+M6+M7+M10+M14	L5
F-0043	gitee	1	M2+M4	L3
F-0044	gitee	1	M2+M9+M10+M12+M14	L4
F-0046	gitee	1	M2	L4
F-0047	gitee	1	M1+M2+M4	L4
F-0048	gitee	1	M6+M10+M12	L4
F-0049	gitee	1	M3+M4+M6+M12	L4
F-0050	gitee	1	M7+M10	L4
F-0051	gitee	1	M3+M9+M14	L4
L3-UNRECORDED-GITEE	gitee	2	M4+M14	L3

(54/74), M13 (29/39); their frequency figures are grouping-driven while their presence is not.

By level. The grouping contributes 42 of the 74 L5 rows and 12 of the 44 L4 rows, and no L3 rows; the split coincides with the grouping-basis split below: the twelve L4 rows are exactly the rows outside the jsbn construct. At key granularity, where a key's standing is the highest level among its packages (Table 14), it is 1 of the 27 L5 keys.

By screen. As recorded in the candidate union, 46 of the 54 grouping rows are literal-only, 5 structural-only, 3 structural+use, and none use-only. Among the 70 non-grouping npm rows membership is more even: 37 literal-only, 10 use-only, 9 structural-only, 12 structural+use, 2 literal+structural. These are recorded hit-set memberships, not counterfactual ablations.

By code similarity. the provenance script classifies each coded npm package by token-shingle containment (k = 12) against all 46 sm-crypto releases, with comments and string literals stripped before tokenisation, under thresholds fixed before the census completed (copy at containment >= 0.5; package-pair clustering at containment >= 0.6 of the smaller set, minimum 200 shingles); the thresholds were not independently calibrated. Under this classification, 34 of the 54 grouping rows are classified as sm-crypto source copies and 10 more as sm-crypto API bundles or forks; 10 grouping rows are not code-similar at these thresholds, and 24 non-grouping defect rows are classified as shipping sm-crypto-like code. The clustering splits the 54 rows across 15 similarity clusters, the largest containing 32 of them. A per-row read of those 24 loci records each locus's position and what was actually checked. In two rows the defective construct is present verbatim in the matched upstream release (the 0.2.1 CBC fall-through at `src/sm4/index.js:227-228`; the 0.2.1 signature-hash default at `src/sm2/index.js:78-90`). In two more (U32 and U33) the defective zero-IV default appears in none of the 46 releases as a lexical matter, and releases from 0.3.0 up instead reject a missing IV; that is a difference from upstream, not evidence of where the construct originated. The other twenty loci sit in wrapper or entry files by path and content - a positional observation that does not exclude inherited defects; within this group we grep-checked two hard-coded values (a private key, a constant IV) against all 46 releases, both absent. Applied file-by-file to the 54 grouping rows' adjudication loci, 26 rows have every parsed locus file at shingle containment >= 0.5 against the upstream union, 11 more have at least one such file, 16 have none, and one row's locus is a record pointer, not a file. These are threshold-hit counts only: containment is shingle-set overlap under fixed sampling, not a share of upstream authorship, and how renaming, minification and shared templates each move it is untested. The counts do not show that the matched rows ship the same upstream defect. A line-level follow-up underlines the limit: the construct the grouping's JSBN-RULE rows share, `new SecureRandom()`, appears in all 46 releases, at least four grouping rows ship jsrsasign/KJUR-style files, and one row hitting containment 0.57 ships a structurally different cipher implementation than its matched release - a threshold hit does not establish descent. *By grouping basis.* Three definitions of the set give different memberships: the recorded adjudication grouping (54 rows); the rows classified as sm-crypto-like code (44: 34 source copies plus 10 API bundles or forks); and the rows whose records show the jsbn-1.1.0 SecureRandom hoisting construct (42: 40 carrying the JSBN-RULE record marker in applying phrasing - five rows mention the marker only to rule it out (five packages, names withheld in this preprint) - plus two more reached through a declared jsbn dependency). The 44-row code set and the 42-row construct set share 38 members: six code-matched rows run other entropy constructs, and

four construct rows are not code-matched: one reaches the construct through a declared dependency (package name withheld; the second dependency-reached row is code-matched) and three are marker-positive rows the shingle classification does not match (names withheld). The construct set is exactly the grouping's L5 rows; the adjudication record carries for each of the twelve rows outside it its own frozen-RNG or pinned-seed reproduction of identical keys or C1 with controls (PASS rows in oracle_1/2/7/8/10/13; all twelve stand at L4). The interventions are distinct: removing the recorded 54 deletes the key (63 of 64 keys, 26 of 27 L5 keys and 32 of 74 L5 rows remain); removing only the construct-defined 42 leaves the key standing at L4 with twelve members (64 of 64 keys and 26 of 27 L5 keys, leaving the same 32 L5 rows, because every L5 row in the grouping belongs to the jsbn construct); removing the code-matched 44 leaves 64 of 64 keys, 27 of 27 L5 keys and 36 L5 rows; removing the 34 source copies leaves 64 of 64 keys, 27 of 27 L5 keys and 45 L5 rows. The corresponding npm precision figures (numerator-side removal) are 70/219 = 32.0%, 80/219 = 36.5% for the code-matched set, 82/219 = 37.4% for the construct set, and 90/219 = 41.1%. These are counting scenarios over explicitly defined sets, not re-estimates of any rate. Grouping-majority category shares move but stay majority on the construct-defined set (M6 38/52, M7 17/23, M12 42/74, M13 23/39). These labels describe text similarity and recorded constructs only: they do not verify cryptographic behaviour, shared root cause, or statistical independence, which are different questions. The adjudication grouping, the screen memberships and the code-similarity classification are three views over existing records and static analysis; we report them side by side because they do not coincide, and we do not treat any of them as the natural counting unit.

Precision under counting scenarios is in §6.1 (clearing the grouping moves npm precision to 70/219 = 32.0%; removing it from both sides to 70/165 = 42.4%). Those are scenarios, not re-estimates.

C.9 The 2026-10-26 C21/C23 re-grade (adopted; historical figures preserved)

§3.3 defines L4 as a security property violated on the shipped artefact and L1/L2 as conformance divergences without one. Two calibration tiers map a shown cross-implementation interop failure to L4: C21 (M4: "L4 when a standard C1C3C2 decryptor is shown failing and mode-0 succeeding (control); L3 if interop could not be shown") and C23 (M14: a reachable non-standard SM2 ID with no override, "L3 unless interop failure shown (L4)"). Whether such a shown interop failure is a violated security property is an adjudication question about these rule tiers, not individual rows; this revision adopts the re-decision. On the §3.3 ladder: a shown cross-implementation interop failure with a discriminating control establishes a conformance divergence without a violated security property, so the pure-conformance claims graded under the C21 and C23 tiers re-grade to L2 - the band's defect level (L1 marks permitted behaviour, not a defect; the claims remain adjudicated divergences in their categories); the re-decision is at the tier-rule level, not row by row. The historical record is unchanged: the adjudication ledger keeps the original levels; the re-grade is a dated overlay file, applied by the census generator before N5 and N6 are computed; the before/after figures below are preserved. No new measurements were taken for the re-grade; it is a re-read of the existing record under the tier rules.

The dated layers connect the counts: 152 DEFECT rows (C.11: npm 127, Gitee 25) become 147 at L3 or higher after the C21 re-grade moves five M4-only rows to L2 (U11, U13, U14, U16, U17); 144 are admitted to the confirmed set (U29 rests on a read-only source review; U30 was never executed; U31 left on 2026-10-27, Appendix C.12); and the recorded replay supports 140 of the 144, 14 of those 140 with caveats (Appendix C.7).

Scope, verified per-row. The re-grade touches exactly the rows whose standing the tiers set. Five keys stood at L4 on M4 alone - each a single package on a single frame, oracle PASS with the discriminating control (oracle_8/9/10), none among the 14 replay caveats - and leave the census, the confirmed set and the adjudicated-defect set: F-0045 U11 (Gitee), F-0058 U13 (npm), F-0063 U14 (npm), F-0071 U16 (npm), F-0072 U17 (Gitee); F-0071's record-table level conflict resolves on the record side by the departure. Three rows named the re-graded tiers as their L4 basis in their review notes and re-stand L4 to L3, remaining admitted: U15 (F-0069, npm) and U10 (F-0043, Gitee), whose notes name C21, and U9 (F-0054, npm), whose record names both C21 and C23. U18 (F-0007, Gitee) was considered for the same move and stays at L4: its review aside "(M4 L4 per C21)" is not the level's basis - the record demonstrates M14 (the signing API returns the caller's private key in its result VO and prints d and the nonce k to stdout, the key recoverable from a log line and a signature, with control) and M10 (a tampered ciphertext decrypts to a modified transfer amount without error, with control), demonstrated properties independent of the re-graded tiers. (The 2026-10-26 key-disclosure review raises the row to L5 on the M14 evidence - Appendix C.10: the M10 demonstration is recorded in the adjudication ledger but the census replay did not reach that leg, oracle_1 NOT_REACHED, so the floor rests on the replay-confirmed M14, oracle_9 PASS.) Category assignments are independent of level (Appendix C.7): a departing or re-standing row keeps its recorded categories. Verified unaffected: the seven L5 rows carrying M4 (an L5 standing necessarily rests on another category); three further packages (names withheld in this preprint), whose L4 rests on M12/M1 evidence; the C15 tier itself, whose L4 requires frozen-RNG reproduction of identical keys/C1 - a predictability property, a different evidence class; and every M14 coding resting on a demonstrated property or a byte-exact defect model (nine packages - names withheld in this preprint - and

set	before	after (2026-10-26)
N6 adjudicated-defect rows (of 270)	152 (npm 127, Gitee 25, Maven 0)	147 (npm 124, Gitee 23)
N5 confirmed pairs	150 (npm 127, Gitee 23)	145 (npm 124, Gitee 21)
census keys	69	64
replay-supported (14 caveats)	146/150 (97.3%)	141/145 (97.2%)
npm precision (N5/219)	127/219 = 58.0%	124/219 = 56.6%
Gitee precision (N5/38)	23/38 = 60.5%	21/38 = 55.3%

standing	before (npm/Gitee/keys)	C21 only	C21+C23 (adopted)
L5	18 / 7+1 / 26	unchanged	unchanged
L4	26 / 11 / 37	22 / 7 / 29	21 / 8 / 29
L3	5 / 1 / 6	6 / 3 / 9	7 / 2 / 9
all	49 / 19+1 / 69	46 / 17+1 / 64	46 / 17+1 / 64

the F-0028-class rows; one further package's C23 coding is an L3 component). Contract-defeat rows and rows with an independent property basis stay on the existing criteria.

U6 (F-0032). The re-grade does not reach this row's level: its L4 rests on M12, and the record carries a frozen-clock reproduction of an identical private key and signature with controls (oracle_7, PASS); the merged note's "brute force not demonstrated (tick gaps)" bounds L5, not L4. The re-graded M4 claim stands at L2; the M14 note records contract defeat with no shown interop failure. The row stays at L4; its key stands at L5 on other packages regardless.

Adopted sets and figures (before to after).

The last column is the 2026-10-26 state. On 2026-10-27 U31 left N5 (C.12): N5 144 (npm 124, Gitee 20), replay-supported 140/144 (97.2%), Gitee precision 20/38 = 52.6%; N6 and the 64 keys are unchanged. The 145 in the table is the figure before U31 left N5 (C.12); the 144 elsewhere is the same set without that row.

Level distribution of the census keys (Table 3), before to after (the C21-only column is kept for the record; C21+C23 is adopted):

The historical series are untouched: the v1 28-finding cohort, Table 19, the first-pass N5 and its precision (C.11), the oracle-outcome census of C.11 (149 PASS over the original 152 rows - a recorded-outcome set, distinct from the 140/144 replay-supported figure over the admitted set), the F-0038/F-0040 L5s (pending at the re-grade; affirmed in Appendix C.10), U19, U12, and the two N6-only rows without a census key (U29, U30). The Maven frame is unchanged throughout.

Open items, listed separately. Contract-defeat wordings (U20's M14) - re-expression open, levels untouched. F-0038/F-0040's L5s and U18's M14 were resolved by the 2026-10-26 key-disclosure review (Appendix C.10): the first two affirmed on the recorded evidence, the third raised to L5.

C.10 The 2026-10-26 key-disclosure L5 review (adopted; recorded evidence)

A record-based review of the three nonce/key-disclosure rows - F-0007 (U18), F-0038 (U7) and F-0040 (U8) - against an independent adjudication of the archived evidence. The review executed nothing: the recovery-probe class remains halted, and every conclusion rests on the record. For these key-disclosure rows the review reads §3.3's demonstrated consequence through the example §3.3 itself names - a usable private key on a live path: the record shows, on the shipped artefact and with a negative control, that an attacker with a stated §3.4 capability obtains the true private key or plaintext; subsequent use of it is not required. This is a reading for this row class, not a redefinition of the ladder: existing L5 rows resting on other demonstrated consequences (forgery, IV-xor chains) are untouched. Requiring use would downgrade the roughly two dozen existing L5 rows that rest on recovery with control and no use step (U47 and eight further packages whose names are withheld in this preprint).

F-0038 and F-0040 affirmed. Both stand at L5 on their original recorded verdicts: the shipped signing code prints the ephemeral nonce k to stdout (loci recorded in the private archive; the archived tarballs re-read statically at these loci), and the recorded recovery of the private key from a log line and a signature, with control, carries the level. Both are labelled "affirmed on the recorded evidence; the probe class was not replayed" and, having no census replay, remain outside the 140 replay-supported pairs.

F-0007 raised L4 to L5 on M14. The census replay (oracle_9, PASS) records two fresh keys, the signing process printing the ephemeral nonce k to stdout (`SM2Factory.sm2Sign`, line 131 of the archived copy), and the private key recovered from a log line and the signature equalling the true key in both runs, with a k+1 negative control. That is the standard's content: an observer of the process output obtains the true key. Two recorded facts do not carry weight: the result VO returning the caller's own key is visible only to the caller that supplied it, and the static `userD` print (`SM2VerifySign.java`:42) was not demonstrated reachable. The raise rests on the stdout-nonce leg alone.

Attacker capability, stated once for all three rows. Each needs the §3.4 observer - a reader of the signing process's stdout or logs - not a public-data reader, and §3.4's "stops at the artefact" applies equally to all three. The category coding differs for the same mechanism: M6 on F-0038/F-0040, M14 on F-0007 (whose M6 folded into M14 at coding time). The historical coding is preserved and the equivalence recorded here; nothing is recoded. On the dual-source discrepancy: the adjudication ledger records an M10 demonstration with control, while the census replay did not reach that leg (oracle_1, NOT_REACHED); the row's floor rests on the replay-confirmed M14 only.

Mechanism and figures. The dated overlay file applies af-

ter the re-grade overlay; the adjudication ledger is untouched. Program-derived deltas: Table 3 keys move to L5 27 / L4 28 / L3 9 (Gitee column 8/7/2); Table 15's F-0007 stands L5; Table 16's Gitee cells move to 6 / 7 / 10; the row-level totals behind C.8 move to 74 L5 / 44 L4. N6 (147), N5 (145 at that date; 144 after C.12), the 64 keys and the replay-supported 141/145 are unchanged by this review: the oracle_9 demonstration is the same recorded evidence the raise grades, and F-0007's level-setting evidence was replayed at census time. A 2026-10-28 local comparison narrows this limit: the archived copy's twelve files match commit fc210dce's tree - ten byte-identical after line-ending normalization, two carrying documented probe-enabling edits about the absent hutool dependency (an import stripped from SM3Digest; Util4Hex replaced by a documented stand-in supplying only bytesToHexString after r, s, Z and e are computed, changing no cryptographic behaviour) - and both defect-locus files (SM2Factory, including the cited `sm2Sign` line; SM2VerifySign) are byte-exact. The edits belong to the adapter-probe build: their compiled classes survive in the research archive (Util4Hex.class is the stub's bytecode, with no hutool reference). oracle_9's recorded compile line names the shipped originals with hutool-all 3.0.9 on the classpath; under the content match above those originals are fc210dce's files, so the archived copy is consistent with the documented probe-enabling copy either way, but no per-file build log binds the oracle_9 run to exact source paths, and the twelve files have been unchanged upstream since 2018-09-03, so the content match corroborates fc210dce without distinguishing it from any commit since then; the pinned SHA itself still rests on the adjudication ledger record.

C.11 Post-hoc re-adjudication history and record lineage

Audit history and independence.

The current counts are those in §5 and Table 3. The earlier 152-DEFECT re-adjudication record predates the adopted re-grades; its PASS/FAIL/blocked labels are attempt records, not a second current census. This appendix preserves that history, the original narrower first-pass result, and the post-hoc calibration rules. Neither the v2 re-adjudication nor the current census has an independent human second pass; older automated agreement figures do not supply one.

The following is historical audit context, not an additional current census.

After the original evaluation we re-coded all 270 candidates against the full M1-M14 checklist, with calibration rules C1-C26 added after the inter-rater study (the JSBN rules are post-IRR additions). Coding and review were LLM-assisted; no human second rater exists. Every DEFECT row has a recorded oracle outcome or attempt record - not necessarily a completed re-run (records: the adjudication ledger and the per-row oracle files; their 2026-10-25 commit window dates the records, not the executions): 149 PASS (U8 and U7 stand at L5 on their original recorded verdicts, affirmed, Appendix C.10, and are not credited as verification evidence), 1 FAIL (U19: the claimed off-curve-C1 property did not reproduce - a one-bit leak did - downgraded to L3), 1 BLOCKED (U12: the cited commit's CA source file is absent from the snapshot, which confirms only the shipped CA key's presence; L4), 1 not runnable (U30: no .NET toolchain; L3). The re-adjudication finds 152 DEFECT rows (56.3%): npm 127/219, Gitee 25/38, Maven 0/13 (per-frame levels in Table 16, Appendix D; the 2026-10-26 re-grade, Appendix C.9, supersedes the affected levels: 147 rows). The two rates are not the same quantity: the original used a narrower (partly M3-scoped) definition, the re-adjudication the full checklist plus later rules. The first-pass N5 (17, 6.3%) and precision are preserved, not replaced. Most of the increase is M12, M14, M6 and M13 rows. Agreement (pre-re-adjudication scheme; Appendix F, T11): six-coder Fleiss kappa 0.657; Kimi K3.1 Cohen kappa 0.569 against the key. The re-adjudication itself has no second pass. The CVE holdout (n = 4, one library; nominated 4/4, localized 0/4, oracle 1/4, then 4/4 after advisory-informed probes; records in the research archive). Record identifiers elided from the body: the sm-crypto re-verification of §3.1 is R-0033; the Maven bundling count of §3.2 is R-0003; the Gitee population reconstruction of §4.1 is R-0002; the use-screen tarball anchors of §5.3 are R-0030; the Gitee Java classification of §5.4 is R-0032; the two-screen union measurement of §6.5 is R-0027 and R-0028, citing the unpreserved per-artefact R-0026; the wrapper-constants re-measurement of §6 is R-0038; the cap analysis of §6.5 is R-0036; the seeded Java defects of §6.5 are R-0037; the detector runs of §6.6 are R-0031 and R-0034; the refuted seeding prior of §6.7 is R-0009. All are in the research archive.

The previous detailed §5 opening is retained here to make the exceptions and retired identifiers traceable without interleaving them with the headline count.

We organise results by what the recorded evidence establishes, not by severity. Table 3 is the v2 census: of the 147 adjudicated rows, 144 package-frame pairs - the 64 finding keys of Tables 14 and 15 - form the confirmed set (N5): the package-frame pairs admitted under the §3 eligibility rule on their recorded evidence, which is not uniform (C.11). The recorded re-run reproduced the level-setting evidence for 140 of the 144 (14 with caveats listed in Appendix C.7); U8 (F-0040) and U7 (F-0038) stand at L5 on their original recorded verdicts (affirmed, Appendix C.10); U19 reproduced weaker evidence than claimed and stands at L3; U12 (F-0047) retains L4 on its original recorded execution (negative control, cited-commit tree); that tree is absent from the archived snapshot (static confirmation of the shipped CA key's presence only). U29 (a read-only source review), U30 (never executed) and U31 (Appendix C.12) form no census key of their own (U31's key keeps two packages); they count in N6, never in the numerator. The census is generated by a script from the adjudication ledger with the adopted 2026-10-26 overlays

(Appendices C.9, C.10). The recorded re-run covered specific disputed categories, not every category of every row. Platform, Maven-frame and library observations (F-0001, F-0002, F-0008, F-0009, F-0015, F-0016, F-0018, F-0019) are narrated in §5.1 and are not part of this census; retired v1 ids: F-0005 (retired in v2: re-measurement of the same artefact under F-0073 keeps the M4 claim and adds newly evidenced M1 at L5; subsumed with evidence preserved; F-0072 is a distinct instance on another artefact, not a merge), F-0009 (known-defect validation case, narrated in §5.1), F-0015 (permitted, L1), F-0016 (no caller on the shipped artefact), F-0018 (folded into F-0014), F-0019 (correctness, not misuse); one further v1 record was withdrawn as NOT_DEFECT and is not cited by id. Each record keeps its reproduction command and boundary.

C.12 The 2026-10-27 admission review: U31 leaves N5

The §3.3 rule needs an executable result on the shipped artefact, and the census already kept U29 (read-only) and U30 (never run) out of N5 for that reason. A review of the 18 sentinel packages against the same rule found one more row in that state. U31 (Gitee, PHP): the record is a source read of its SM2 and cipher source files (a fixed ephemeral scalar enabled by default) and an independent point calculation showing that the scalar matches the published point (`oracle_1.csv:13`, PASS). The shipped PHP was never run (no PHP in the sandbox), and the negative control for the default path was a source read. The arithmetic is executable; the claim that the shipped default path uses the constant is not. It remains an adjudicated L3 defect row (N6) and is no longer counted in N5. Nothing was rerun, and the adjudication ledger and the oracle records are untouched; the change is one entry in the census generator.

Effect: N5 145 to 144 (Gitee 21 to 20), replay-supported 141/145 to 140/144 (97.2% in both), Gitee precision 21/38 = 55.3% to 20/38 = 52.6%, Gitee sensitivity 21/44 = 47.7% to 20/44 = 45.5%, Gitee N5/N6 21/23 to 20/23; N6 (147), npm figures and the 64 keys are unchanged (L3-UNRECORDED-GITEE keeps U37 and U38). The other 17 sentinel packages have a shipped-artefact execution in their oracle record (16 PASS, and U19, whose FAIL only rejected the L4 claim and reproduced a one-bit L3 result); 5 of the 17 probes (five packages, names withheld in this preprint) cannot be mapped to an archived script, a traceability gap in the private archive, not a failed result. Reviewers cannot inspect either (Open Science).

Appendix D. The Use Screen

the use-screen script judges an artefact by use: an import, an `#include`, or a call or typed symbol of a Guomi library. It was run over the 679 npm packages the literal screen scored zero, the only population on which a third screen can be evaluated, and named 66 candidates. Three of its own defects were found before adjudication. A dependency pattern that allowed any run of characters before `sm-crypto` matched `wasm-crypto`, admitting a blockchain library on a substring: an unbounded run before a token matches the token inside a longer name. Its real false-positive class is a receiver named `sm2` or `sm3` that is something else (SoundManager2, a `StringMap`, a SuperMemo-2 scheduler). And five packages matching `SM2.point(` (`U8` and four forks) were first read as an international curve object; their `lib/sm2.js` defines the GB/T 32918 curve, so they are Guomi implementations and that reading was withdrawn.

Re-adjudication detail. Table 16 gives the per-frame outcome of the full M1-M14 re-adjudication.

Adjudication time points. The 66 use-screen candidates were adjudicated twice. The screen's own pass (2026-10-23, scoped to the nominating evidence) confirmed seven defects; all seven remain DEFECT in the census and map to five keys (F-0025, F-0026, F-0027, F-0029, and F-0028, three of whose six packages were the confirming rows, the other three recorded with literal or structural membership). Of the census's 10 use-only DEFECT rows, five sit under formal finding keys and five do not: one each under the IRR-18 and IRR-46 disclosure records and three under the L3-UNRECORDED-NPM sentinel, which is a census accounting row, not a finding key. Three of the 10 are those confirmations, one was adjudicated conformance, one below-threshold, and five clean in the earlier pass; the difference is the full-checklist re-read, not a re-measurement. The row-level mapping is regenerable: a generator script writes the mapping from the two records.

Seventeen candidates import or `#include` a Guomi library and none trips a misuse pattern; two that looked as if they should have tripped M2 (two libraries, not named here) are libraries that expose the mode as a parameter, which F-0005's rule already clears. The remaining 44 are calls or typed symbols without an import, satisfiable by a vendored copy, so each had to be read; all were. One result shows why judging by use matters: `cryptopp.cxx` is Crypto++, an international library, and it ships `#include "sm3.h"` and `#include "sm4.h"`, found by neither name nor data. Every verdict is written as role, layer, verdict, category, reason and anchors, and the adjudication script fails if an anchor substring is no longer in the tarball.

Appendix E. Denominator Decisions

On-topic bound. Measured before the use screen, at the 230-candidate three-frame scope of that point, 182 candidates carried Guomi by data and 48 did not (28 npm, 12 Maven, 8 Gitee). Reading those 48 showed that 25 use Guomi by delegation, so the on-topic denominator was 207 of 230 (90.0%) and precision against an on-topic denominator would have been 5.8% at that scope. The paper divides by every candidate instead, because the assumption that an unread off-topic

Table 16. Per-frame outcome of the full M1-M14 re-adjudication (C.11), re-graded per Appendix C.9 and C.10. The DEFECT percentages are N6/N3, not the N5/N3 precision of §6.1. The original figures are the historical 2026-10-25 table (per-frame rows): npm 127 (21/42/64), Gitee 25 (5/11/9), total 152 (all-frame rows: 26/53/73).

frame	candidates	first-pass N5	re-adjudicated DEFECT	L3 / L4 / L5
npm	219	14	124 (56.6%)	23 / 37 / 64
Gitee	38	3	23 (60.5%)	6 / 7 / 10
Maven	13	0	0	-
total	270	17 (6.3%)	147 (54.4%)	

candidate is clean is exactly what §6.5 found to be wrong.

Unopened candidates. A sweep found 146 candidates without any record; two were defects (F-0022, F-0023). Of five recorded as unopenable, four were decidable once an extractor bug was fixed (the files had been on disk throughout), one of them `U48`, now F-0024; the fifth, a minified bundle, was attributed by construct (its one match is an API parameter). The candidate file is empty and kept.

Search terms. Thirteen npm search terms are recorded as DNS-failed at discovery time (a list reconstructed by recall five days later, R-0022; provenance limits under Discovery term records below). Re-run with retries, all resolve and return 168 names not in the corpus, many of them general infrastructure (cloud SDKs, a Redis adapter) that mention SM2 only in passing. Adding them would enlarge the denominator with 168 unopened candidates, so they are reported as a bound on N1 and not added.

Discovery term records. The study-time record states 11 + 39 npm search terms (Table 2, §4.1), matching the wave-2 commit message ("381 packages from 39 more search terms", c78eeab79); the script committed in that same commit enumerates 11 + 47 (the discovery script, term lists unchanged since), and the run artifact tags 15 hit-producing wave-2 via-terms - zero-result terms leave no trace, so the executed set cannot be enumerated from committed records. The 13 DNS-failed terms above carry no committed provenance: study-time evidence is a count only ("about 14", same commit message), the named list first appears five days later (ebabf6637), and R-0022 records that the failures were printed but not persisted and the list was reconstructed by recall; 12 of the 13 appear in neither scripted wave, so the failure list and the script waves compose into neither 50 nor 58. Gitee's records disagree similarly: the corpus script lists 11 queries while the run artifact records 10 executed (SM4 absent, SM3 failed at run time). Maven's six `fc:` queries are recoverable from the artifact's via-fields. The discrepancy is unreconciled: neither 50 nor 58 is claimed as the true count; the temporal-holdout design records the gap and freezes the scripted lists prospectively.

Gitee structural pass. Over the 45 repositories on disk at the time, the structural screen named 10 Guomi-carrying repositories that the literal screen did not; four were checked by reading the file and are true positives. That pass walked files differently from the frame's screen, so the frame's N3 was not restated from it; the frame was later re-screened over all 59 clones with the shared scope.

Appendix F. Threats to Validity

T1 Oracle coverage. SM2, SM3, SM4 and SM2 encryption are all calibrated (§3.1). The F-0010 and F-0017 probes were initially run with third-party SM4 implementations; no record of a re-adjudication with the oracle's own SM4 survives, so that step is not claimed and the two verdicts rest on the third-party implementation's correctness as evidenced in their finding records. **T1b Baselines.** The Guomi-extended Semgrep rules were written by us, once, and scored on defects we also wrote (§6.6); no Guomi-extended Java configuration was evaluated. CogniCrypt and CryptoGuard cover Java only (§7). **T2 Impact.** Verdicts stop at the artefact; downstream is measured where possible and reported with its scope (§7). **T3 Recall.** Population recall is not measurable; precision is reported at N5/N3. Recall on seeded defects (§6.5) depends on a mix we chose and on an oracle written by the same author as the mutants. Clearance adjudications got one bounded check: under a frame frozen before any sampled unit was read (estimand: false-clearance re-review of the 118 adjudicated non-DEFECT rows, not the screen miss rate and not recall; strata npm 47/45 at the 20993-byte tarball median plus gitee/maven censuses; seed 3040739601554316973; the run record is in the research archive), 56 units were re-audited, 48 at full and 8 at partial coverage with the visibility limit named per unit (a native-code IV, a process-global-state scenario). The raw count of clearance errors found is zero per stratum; two named items stayed statically undecidable. The auditor is the same single LLM-assisted author, so this is not an independent review, the zero is a sample count and not a correctness proof, and the 8 partial-coverage units and 2 undecidable items are not cleared by it. **T4 Maven.** "No application-layer Guomi in this frame" is stated as "we found none"; two independent methods agree on the frame's single Guomi artifact (§6.3), and the Gitee Java classification shows Java applications exist outside this frame (§5.4). **T5 Screening coverage.** Quantified in §6.5 (15.8%, interval 8.5%–27.4%) and corrected by re-screening; encoding coverage is synthetic (Appendix B.2). **T6 Gitee reproducibility.** Fourteen working trees had been removed to reclaim disk; all were re-cloned from recorded URLs and the frame is measured over 59 of 59. **T7 Standards reading.** The arrangement rests on GB/T 32918.4-2016, held as clean text, where `C1‖C3‖C2` is stated in step A8, in de-

cryption, and in the worked example our implementation reproduces. GM/T 0009-2023 has no text layer (fourteen page images); its §7.1-7.4, §8.1 and §10 were read from the images and are recorded verbatim with page references. GM/T 0003.4-2012 sections 6.1 (A1-A8) and 5.4.3-5.4.4 were read first-hand from the official-source page images on 2026-10-12 and are recorded verbatim with page references (pages 8 and 10 are JBIG2 and remain unread). GM/T 0010-2023 remains an OCR transcript; it supplies message-syntax guidance only, and the arrangement claim does not rest on it. **T8 V8 seeding.** Twenty-four independent Node 24 processes produced 24 distinct first `Math.random()` outputs, as did a repeat on Node 22.23.3 (Linux), and 100,000 draws in one process produced no repeat; the latter bounds state size, not entropy per draw. **T9 Disclosure.** Incomplete at the time of writing (Ethical Considerations). **T10 Related-work coverage.** A forty-two-entry matrix was built from sources that could be opened, eleven entries from abstracts only (Appendix H). **T11 Rater agreement.** The second coder is an LLM agent applying the written rules; its κ is a check on the rules' decidability, not on human judgement (§6.4). Six further LLM passes (Claude Opus 5.5, Muse Wonder, MiMo-V2.6-Pro, DeepSeek V4.1 Flash, Step-5 Preview, Kimi K3.1 Pro; fresh sessions, same package) scored κ 0.31–0.66 against the revised key (four at 0.64–0.66; Kimi K3.1 Pro at 0.57; Step-5 Preview, whose extra calls were mostly jsbn `SecureRandom` flags, at 0.31) and Fleiss κ 0.657 among themselves, erring toward DEFECT. Both agreement values predate the C1-C26 and JSBN rules and therefore measure the earlier coding scheme; the full-checklist re-adjudication has no second pass and no agreement measurement. Adjudication of their extra calls added seven packages to the confirmed set (IRR-02/-33, one combined record covering two packages, plus IRR-18, -32, -34, -46, -55: built-in SM4 keys, imposed ECB, constant or zero IVs), which the literal half had cleared on the M3 identity rule alone; all seven are counted in the totals and their disclosure state is tracked separately (Appendix G). One further call, IRR-40 (`vue-msm4`), was reversed to NOT_DEFECT on three independent records: its only export is a self-contained demo widget and the fixed IV protects no caller asset; a retraction of its disclosure was sent on 2026-10-26 (Appendix G).

T12 Structural-detector generality. Per-file records behind the Recall limitation (§7): the Gitee rescreen records structural Guomi data in U18 (SM2; Java), U44 (SM3, SM4; Java), MySuiteA (SM4; C) and sm234 (SM2, SM3, SM4; PHP), with per-file verification archived; the file-level false negative is U44 `SM4Constant.java`, a (byte)-cast T-table, inside a repository-level true positive. The stress battery (author-designed synthetics; battery and per-file hashes archived in the research archive, not part of this checkpoint) covers ten transformation classes over SM2/SM4 constants - T-table split, (byte)-cast, deinterleaving, runtime generation, base64, literal escapes (octal, unicode), word packing, word-array SM2, reversal and fromCharCode encoding; nine defeat the detector, only fromCharCode-encoded tables are caught, and all six negative controls (SHA-256 K, AES S-box, secp256k1 n, P-256 n, an ascending table, CRC-32) stay silent.

Artefact provenance recheck (detail, cited from §7). All 124 admitted npm rows stand on registry tarballs frozen by sha256; a records-only recheck on 2026-10-27/28 re-downloaded them byte-identically (the last exception, U61, unpublished after our analysis, was recovered from two byte-identical archive copies and its runtime claim replicated offline with throwaway keys, a narrowed-window replication rather than an exact re-run; recorded in the research archive). Gitee provenance is weaker: six admitted rows re-verified at their recorded SHA, one rests on a date/history inference, four on vendored copies without a commit pin and nine on unpinned HEAD-consistency, which shows presence at check time only (Appendix I). Verification depth differs by level: the 37 admitted L4 rows (plus 5 retained checks) received semantic line-reads, while all 64 L5 rows, 23 L3 rows and 20 Gitee rows received presence-level checks only. The recheck re-ran no runtime oracle and is not independent validation: 142 of 144 admitted rows carry a recorded PASS (two L5 standings rest on original recorded verdicts, Appendix C.10), one row stands at L3 after its L4 claim failed to reproduce, and one stands at L4 on a historical execution whose snapshot lacks the cited file. A bounded re-execution of the 27 rows lacking a concrete shipped-path token against their frozen tarballs recovered every victim value, controls retained, so all 64 L5 runtime claims are artefact-bound.

Appendix G. Disclosure Record and Channel Availability

Table 17 records what has been sent.

The repository tracker records nine further sends dated 2026-10-24: the 23:51 batch - IRR-02/IRR-33 (U23, one key under two registry names, one letter to the two registry-listed maintainer addresses), IRR-32 (U21), IRR-34 (U24), IRR-40 (`vue-msm4`) and IRR-46 (U25) - the 23:58 batch - IRR-18 (U22) and IRR-55 (U26) - and two out-of-frame letters, F-0030 (U27, Gitee, to the commit author) and F-0031 (U28, NuGet, to the source-repo commit author). Table 17 does not list them because delivery could not be verified; per-letter state: the original sent messages are preserved in the sender's mailbox for all nine; the five whose originals had not been located were found in the provider's sent records on 2026-10-27. Provider sent records confirm sending, not delivery; an absent bounce is not evidence of delivery, and no bounce for these recipients is on record. IRR-40's package was subsequently adjudicated NOT_DEFECT (Appendix F, T11), and a retraction covering the letter as sent (fixed SM4 key and IV) was sent on 2026-10-26.

GitHub private vulnerability reporting was checked on 2026-10-23 for every repository involved and none can take

Table 17. Disclosure state as of 2026-11-01; refreshed again before submission.

finding	artefact	channel	status
F-0014 (+F-0015)	U44	email to the POM developer	sent 2026-10-11, awaiting first reply
F-0016	same artefact, its TLS server class	same thread	queued after the first reply
F-0017	U45	email to the four maintainers in npm metadata	sent 2026-10-12; body delivered, attachment blocked
F-0003	U46	email to the npm maintainer	sent 2026-10-12, awaiting reply
F-0021	U47	email to the top-level npm maintainer	sent 2026-10-14, awaiting reply
F-0024	U49, U48	email to the project's official address	sent 2026-10-23
F-0022 + F-0029	U50 (and two others), U51	one email to their shared npm maintainer	sent 2026-10-23
F-0023	U52, U53	one email per package to its npm maintainers	sent 2026-10-23; a correction narrowing the U53 letter was sent 2026-10-26 (its fallback is unreachable in the normal flow; the package's M9/M12 findings stand)
F-0025	U54	email to the npm maintainer	sent 2026-10-23
F-0026	U55	email to the npm maintainer (upstream archived)	sent 2026-10-23
F-0027 + F-0011	U56, U57	one email to the publisher's three addresses	sent 2026-10-23; one address hard-bounced (550), two did not
F-0028	U58, U59, U60	one email covering all three packages, to the three registry-listed addresses	sent 2026-10-23
F-0009	bcprov 1.57–1.58	public question to the maintainers (not a vulnerability)	maintainer replied: known (#218), fixed in 1.59 [34]

Table 18. Disclosure-channel availability over the 794-package npm corpus (793 fetched successfully).

channel field present	packages	share
any of repository / bugs / homepage / maintainer email	793	**100.0%**
a maintainer email	790	99.6%
repository	604	76.2%
homepage	568	71.6%
an author field	570	71.9%
bugs	552	69.6%
maintainer email is the only channel	**178**	**22.4%**

a report: it is disabled for four (U49, the U52 repository, one repository and bc-java), one is archived (the upstream repository), and the other packages declare no repository. For F-0014, the repository offers no private vulnerability reporting and its mirror only public issues, so the POM address is the only private channel; for F-0017 the package has no repository, homepage or bugs URL. One F-0017 letter carrying a reproduction archive was rejected by the recipient's content policy after the recipient was accepted, while the plain-text body was delivered. Since then the evidence goes in the body of the letter and archives are supplementary, and channel usability is treated as two checks: the address resolves, and a message carrying the evidence is accepted.

Table 18 measures the first check over the npm corpus from public metadata only.

Zero of 793 packages lack every channel, so the absence of an address is not what makes disclosure hard here; for 22.4% mail is the only route. npm's top-level `maintainers` list and the version manifest's list disagree for 69 of 793 packages (8.7%), in both directions, so tooling must say which it reads. None of this establishes that any address receives mail; delivery was not tested.

Letter log through 2026-10-27 (detail, cited from Ethical Considerations; status after that date is in Ethical Considerations). Disclosure letters have been sent privately by email to the addresses the maintainers publish in their registry metadata or, for U49, its official address: F-0014 with F-0015 (2026-10-11), F-0017 (2026-10-12), F-0003 (2026-10-12), F-0021 (2026-10-14), and on 2026-10-23 F-0024, F-0022 with F-0029 in one letter, F-0023 (one letter per package), F-0025, F-0026, F-0027 with F-0011 in one letter, and F-0028 (one letter per maintainer). F-0016 is queued inside the F-0014 thread. On 2026-10-24 the IRR-32 letter (U21: imposed SM4-ECB in the envelope) was sent to the address the maintainer publishes; on 2026-10-26 a withdrawal was sent for one premature claim (vue-msm4) and a correction for one letter (U53); on 2026-10-27 a correction was sent in the U21 thread retracting one unsubstantiated claim (the envelope ciphertext-order assertion; the SM4-ECB observation stands). GitHub private vulnerability reporting was available for none of the repositories involved (Appendix G). Coverage is partial and stated exactly: on 2026-10-27 letters were sent to the maintainers of the 42 previously undisclosed census keys (F-0004, F-0006, F-0007, F-0010, F-0012 and the admitted keys F-0032-F-0073) and of the two L3 sentinel keys (17 packages), so every census key except F-0013 (whose key a third party had already published) is now covered by at least one letter to an address the maintainer publishes; sixteen of those 42 keys stand at L5 (27 census keys stand at L5 in all; Tables 14 and 15). Sent is not delivered is not remediated, and a key covered by a letter is not

a key whose every claim is notified. The remainder that still counts as undisclosed has three distinct sources. First, current campaign items: one package in the F-0032 lineage whose letter is held after two failed submission attempts, and the one F-0040 claim, a recorded log line, that the F-0040 letter deliberately excluded pending its independent re-review. Second, recipient failures observed by the last targeted mailbox check (2026-10-27; later bounces are in Ethical Considerations): two F-0032-lineage packages whose single registry-published maintainer address does not exist - the letter reached no one and the registry lists no alternative contact; one F-0032 package for which one of the eight published addresses returned mailbox-not-found; and partial-route failures on four further letters (three of seven addresses for one npm letter, and one of two addresses for each of three Gitee letters), where the remaining addresses returned no failure notice, which is not evidence of delivery, so their coverage is unverified. Third, the historical sent-but-unverified rows listed below (IRR-18, IRR-33, IRR-34, IRR-46, IRR-55). Also on 2026-10-27: a supplement was sent in the U21 thread covering the further finding classes not named in the 2026-10-24 letter; letters were sent to the maintainers of the three later-found F-0028 family instances (letters had been sent to the paper's three F-0028 packages on 2026-10-23); and letters were sent to the four remaining Gitee repositories, one addressed to the upstream maintainer address published in the mirrored project's package-registry metadata. F-0013's key was published by a third party on 2026-08-13, before our letter window. Five further rows (IRR-18, IRR-33, IRR-34, IRR-46, IRR-55) have letters recorded as sent on 2026-10-24 whose delivery could not be verified (Appendix G), so they count as undisclosed here; the merged IRR-02/IRR-33 key stands at L5. In this preprint, artefacts with unnotified findings, and those notified whose window is still open, are pseudonymized (U1-U61) and identifying versions and code fingerprints generalized; the mapping is retained in the private archive. Any restoration of names at camera-ready follows the recorded per-letter terms: 90 days from the first notice for a silent maintainer, an agreed date where there is one, and current public status. Completing disclosure or anonymizing those names is our standing pre-publication risk-control requirement, not a venue rule (Appendix F, T9). Each letter carries its evidence in the body (artefact coordinates and hash, the measured behaviour, a negative control, the standard or contract text, the affected versions, what we do not claim, and questions, not a verdict) and asks for a 90-day window. Replies are stated as of the disclosure-mailbox check of 2026-10-29; later developments are in the status paragraph of Ethical Considerations. Besides F-0009 below, one maintainer reply had been received: the maintainer of U61 (an F-0032-lineage package, letter sent 2026-10-27) replied the same day that the package has been unpublished from npm and is unmaintained, and the npm registry independently returned 404 for the package the same day. The analysis in this paper ran against the version published at analysis time; the package's subsequent removal changes its current distribution status, not the historical record. Replies arriving after that check are not reflected in this revision. Table 20 (Appendix J) collects the maintainer-side responses recorded up to 2026-10-28; it is the only evidence in this paper not produced by our own adjudication. Four maintainers have replied so far; we report no reply rate, because the letters are between 2 and 18 days old and delivery is unverified for some. F-0009 is not a vulnerability, so it was posted as a public, non-security question to the library's maintainers. A maintainer replied that it is the known issue #218, fixed in 1.59 [34]. Findings graded as permitted behaviour (F-0002, F-0015, F-0008's 1.59+ half) or as divergence without a security property (F-0001) are reported as such and not disclosed as vulnerabilities.

Appendix H. Literature Search Procedure

Stage 1 used general web search and arXiv over the misuse-measurement, SM implementation and JCA themes, plus a direct search for a Guomi misuse measurement. Stage 2 used ScienceDirect through an institutional library subscription (R-0010; the institution is not named for anonymity). Every query was submitted and observed independently, and each zero was re-run up to three times to separate a genuine empty result from a form failure:

```
ShangMi OR Guomi cryptography misuse ecosystem    2   (both use SM
↪  as a component)
SM2 SM3 SM4 implementation interoperability        0
SM2 signature identity binding                     0
Bouncy Castle SM2 SM3 SM4                          0
ciphertext C1C2C3 C1C3C2 SM2                       0
cryptographic API misuse empirical study ecosystem 0
--- controls (the search does return results) ---
SM3 hash function                                  3
SM4 block cipher                                   5
cryptographic API misuse                           5
SM2 implementation analysis                        5
elliptic curve digital signature implementation   36
```

Web of Science required a sign-in we did not have. CNKI was reachable but its "no data" wording appears both for zero results and for failed loads, so its null results are not used as negative evidence (R-0011); its one substantive result is the thesis of Appendix C.5 (R-0012). A forty-two-entry matrix then placed this work against the static, benchmark, empirical, dynamic, differential and LLM lines, each entry verified against a full text, a publisher page, a DOI record or an NVD record; 26 were read in full and eleven rest on abstracts only. The novelty claim is therefore "not found under these constraints", weaker than "does not exist"; the positioning against JScamd rests on its full text, which we read. The §2.2 coverage check counted the terms `SM2`, `SM3`, `SM4`, `SM9`, `ZUC`, `ShangMi`, `Guomi`, `GM/T`, `GB/T`, `OSCCA`, 国家密码, 商密, 国密 (zero occurrences) and `AES`, `RSA`, `SHA`, `MD5`, `HMAC`, `PBKDF`, `ECB`, `CBC`, `GCM`, `CTR`, `CFB`, `CryptoJS`, `forge`, `elliptic` (all present). We additionally screened the published artefact itself

(Zenodo 20567432, zip sha256 edc30e14…; rule file src/core/model.js sha256 4bf85531…): its 41 rule definitions (40 iterated) name no SM/Guomi source or sink, and no rule condition expresses identity binding, ciphertext-component arrangement, or curve-parameter provenance; the rule nearest to M5 is a strength bound on named standard curves, a different misuse. The artefact's own crawler-output data list six Guomi library repositories, so the absence is at the rule level, not the data-collection level (no author-intent claim). The covered generic categories bind to non-SM API names, so under the direct-listed-name conditions the implementation does not reach SM-library call sites; this is a binding-scope statement, not an execution observation.

On 2026-10-26 we ran a bounded supplementary search for a prior Guomi-misuse measurement: nine web queries (Chinese and English, verbatim strings archived), site-scoped engine coverage of cqvip.com (article records), and scope reads of the three candidate works that surfaced (one complete-text read, one abstract-plus-references assessment, one full-HTML read). No prior Guomi-misuse measurement of shipped code was found under these channels; the closest Chinese-language works are a hardware cryptographic-core verification study and an algorithm-evaluation survey; the full-text read places the former outside the misuse-measurement class, the abstract-level assessment the latter. CNKI and Wanfang article corpora were not covered (connection failure and a human-verification wall respectively, evidence archived), Scholar and WoS were not reached; this statement is bounded to the enumerated channels and date.

Appendix I. Gitee Beyond the Frame

Table 10's Gitee row was measured before the shared scope gained five languages; on the current scope it reads 31 carrying Guomi and 39 candidates, the difference being `fabric-ca`, whose first 600 files changed with the scope.

The Gitee frame is the 60 most-starred of 354 repositories with strong metadata. We screened all 294 others with the frame's own screen (seed 20261023; R-0036); 280 could be cloned, the rest now require authentication. Membership is 100 of 280 (35.7%), lower than the frame's, as expected for the less-starred remainder, and the screen names 138 candidates under the 600-file cap (155 without it).

Every candidate has a recorded verdict. The 69 M3 repositories use the default identity, which §5.1 treats as permitted. We read all 32 M1 candidates by hand: 28 are standard vectors, algorithm constants or demonstration keys, one literal is a public key, one demonstration client for a third-party gateway carries credentials we did not test, because testing them would mean using them, and two are defects. In U27 the password encoder ignores its key argument and encrypts every password with an SM4 key and IV written in the source, so stored passwords are reversible, and the repository's own seeded password decrypts to readable text (F-0030, L5 source chain). U28 (NuGet) ships a working SM2 key pair behind a default-key encrypt helper, and the private key is present in every published build we checked (F-0031, L4).

Both findings lie outside the three measured frames and are not counted in N5, precision or the abstract. Their 2/138 is therefore not comparable with the frame's 3/38 - the three screen-found application-layer findings (F-0012, F-0013, F-0024) among its 38 candidates, a finding-level count distinct from the package-level precision of Table 10. That they appear at all in the less-visible remainder is the point: the application-layer M1 shape the frame found (F-0012, F-0013, F-0024) recurs where no one is looking. Both were disclosed by email on 2026-10-24, and this appendix prints no key, IV or password.

Admitted-row provenance, per row. The 20 admitted Gitee rows divide into four provenance classes (per-row key, class and basis: reconciliation file in the research archive): six re-verified at their recorded SHA (clone HEAD equal); one, SM, corroborated at a HEAD unchanged since 2022-05-26 matching the corpus `last_push` record - a date/history inference, not a recorded-commit pin, adopted after a wrong-repo flag (three same-named candidates exist) was caught and retracted; four rest on local vendored copies without a commit pin; nine on unpinned HEAD-consistency at clone time. An earlier recheck note's own 12/8 summary line does not reproduce from its per-row itemization (7 pinned, 13 unpinned); the reconciliation recomputes the classes from the roster (reconciliation file in the research archive; per-record overlay in the same archive).

Cap sensitivity (detail, cited from §6.5). An identifier rule matched `privateKey` but not `PRIVATE_KEY`, missing F-0013; its repair briefly matched 56 of 59 Gitee repositories; the shared file scope omitted five languages (Appendix B). Two evasions are adversarial: the permuted key of F-0027, and the XOR split in the `U49` server, on which our literal key rule fires zero times; the server also exceeded the 600-file per-repository cap. It entered only through the mobile client's matching pair, a false negative of the screen, not added to N3. Eight frame repositories exceed the cap: the capped screen names 39 candidates, the uncapped 44; the published 38 predates the October scope refresh - the one-candidate difference (`fabric-ca`) comes from the refresh changing the sorted first 600 files, not the cap. Of the five cap-hidden candidates, three carry affirmative negative adjudications (`TASSL`: a permitted library default identity; `anypay`: an off-topic anti-replay nonce; `U49`: a void-returning RSA `Certificate.verify` with nothing ignored); the other two (`fabricv2.2-gm`, `gmgo`) are structural-only, as is `fabric-ca`: the screen names Guomi data but no misuse category, and no executable oracle ran on them, so they are screened-negative at screen level, not oracle-verified. N5 is unchanged at 20, so Gitee precision moves from 52.6% (20/38) to 45.5% (20/44); we publish the capped frame and report this as denominator sensitivity.

Appendix J. The Static-Tool Baseline (npm Pilot)

Patch propagation (context for §8). Of the 790 readable packages, 41 declare a dependency on `sm-crypto`, and for 40 of them the declared range resolves to a version that still has CVE-2026-73567 open; a further 45 ship a copy of the prefix source (`analysis/patch_propagation`, declared ranges only, no reachability).

JScamd and CrySL, specifically. JScamd's 16 constraints cover four of our categories (Table 1); adding Guomi sinks alone would still give its taint rules no slot for the contract layer (M3, M4, M14) - an argument from the standards, not a tested extension. The public CrySL rule repository (checked 2026-10-23) has no SM rule and rejects the SM algorithm names (§6.6); a constraint answers neither P1 nor P3 without a calibrated oracle. Our novelty is the measurement and the oracle: an extended JScamd or CrySL might find these defects; neither, as published, does. A file-level pilot follows in this appendix. The SM literature studies implementations (side channels [22], fault attacks [23], advisories [24, 35], §3.1); fuzzers (Cryptofuzz [18], CLFuzz [19], survey [20], Wycheproof [21]) test library correctness, not application calls; none covers SM operations or vectors (§6.6).

How do the two closest published static tools fare on the confirmed set of §5? This appendix reports a one-shot, file-level pilot run over the 126 npm packages in that set (pre-regrade numerator) - the 127 minus `U61`, which pins no registry version and declares a dependency on a registry package of the same lineage in the deps.dev index (queried 2026-10-26), manifest evidence of its provenance as a wrapper - scored against the pinned registry tarballs (byte-identical to the registry, verified per package before scoring; scripts and manifests are archived in the full research archive; the present checkpoint does not include that archive. The scoring record and the raw tool outputs are preserved here, and a 2026-10-28 cross-validation (R-0039) recomputed every Table 19 count from them exactly, with one bounded exception: of the 17 recorded CrySL Guomi-name rejections, the kona-crypto cell reads 1 while the raw alerts hold 1-2 candidates (a compound `SM3HMac` key-algorithm name), a selection rule the preserved artefacts do not document).

The four-family baseline run (§6.6), in detail.

Would off-the-shelf tools have found these defects? We ran four tool families, unmodified, over the curated 28-finding cohort: Semgrep OSS 1.182.1 [32] (public `p/java`, `p/javascript`, `p/secrets` and `p/gitleaks` rules) on the published tarballs and sources; and on every Java bytecode target (four published jars, three repositories compiled from declared dependencies), SpotBugs 4.9.8 with FindSecBugs 1.14.0 [33], CogniCrypt_SAST 5.0.1 with the published CrySL JCA rules [2] and CryptoGuard 04.05.03 [3]. Semgrep ran with its default ignore list disabled (`--x-ignore-semgrepignore-files`), so `dist/` and `build/` are scanned. A tool *detects* a finding only when a rule whose meaning is the finding's property fires in the defective file or class (Table 19).

Table 19. Baseline detectors on the curated 28-finding cohort (v1 scope; the v2 census is Table 3, in full Tables 14-15). F-0016 is retired in v2 (no caller on the shipped artefact), so 8 of the 9 detections below remain census defects; re-scoping the detector comparison to the full census requires re-running the tools. "Applicable" counts findings whose artefact the tool can analyse (JavaScript for Semgrep only; jars without source for the bytecode tools only). CogniCrypt's one detection is a "key not properly generated" predicate error, CrySL's nearest notion of a hard-coded key; under a strict reading it has none. F-0028 (M14 and M1) counts in both property rows; its detection is the M1 key.

	applicable	detected
Semgrep	19	7
SpotBugs + FindSecBugs	12	2
CogniCrypt_SAST (CrySL)	12	1
CryptoGuard	12	0
any tool	**28**	**9 (32%)**
Guomi-specific properties (M3, M4, M5, M11, M14)	10	**0**
generic properties other than M1 (M2, M9, M12)	8	**0**

All nine detections are hard-coded keys (F-0011, F-0016, F-0021, F-0022, F-0023, F-0024, F-0027, F-0028, F-0029), the tools' home ground. Their ECB, static-IV and weak-PRNG rules attach to JCA sources and sinks, while the Guomi API surface (a wrapper's `sm4.encrypt(..., {mode})`, a bespoke `SM4Cipher`, keys assembled from `Math.random()` characters, a constant-seeded DRBG) is outside them: consistent with the §2.3 lens but not a test of it: none of these configurations has Guomi rules (the public CrySL repository has no SM rule), so the misses follow from rule coverage, not from the contract's shape. In the other direction, CrySL's JCA rules reject the Guomi algorithm names themselves (17 constraint errors), and three tools flag a compliant SM4 library class (ECB, `STATIC_IV`, 22 CrySL errors); they do not draw §5.2's library-versus-application distinction: six compliant npm artefacts draw no alarm outside README and test files.

The Guomi rule experiment (§6.6), in detail. The rules are in `harness/bench/guomi_rules.yml` and the scorer in `harness/bench/guomi_rules_eval.py`; Semgrep 1.182.1, ignore list disabled. A seeded defect counts as localised when its tree gains a hit of its own category that the clean control does not have (an M9 defect may be caught by the M1 rule). Result 48 of 88: by category M1 9 of 12, M11 5 of 8, M12 9 of 16, M14 5 of 12, M2 6 of 12, M3 4 of 8, M4 3 of 8, M9 7 of 12; by host `sm-crypto` 14 of 32, `sm4-crypto` 13 of 20, `co-fb-crypto` 11 of 20, `we-sm2-password` 10 of 16. In M14 the sign-extension rule catches 4 of 4, the double-hash rule 1 of 4 (split form only) and nothing catches the key embedded in the signature (0 of 4). On the unmodified hosts the rules already fire (`sm4-crypto` 12, `sm-crypto` 8, `co-fb-crypto` 3, `we-sm2-password` 0 hits), so any use outside a seeded benchmark would need triage. The rules were written after the defects by the same author, in one pass with

one syntax-debugging round and no tuning on scores. The result shows what a Guomi-aware rule can express, not how a third party's rules would perform.

Protocol and freeze. The original run's protocol was frozen before any output was inspected (its `FROZEN.sha256`, seed 20261026, stratified sample of 20 DEFECT + 20 NOT_DEFECT + 10 OFF_TOPIC, versions pinned, `npm pack --ignore-scripts`, no lifecycle scripts executed). The freeze is narrower than a preregistration: the cohort, tool versions and the single-package scoring commands are hash-locked, but the driver scripts evolved after outputs were produced and their earlier versions were not preserved. Of the 26 frozen entries covering 25 unique paths, 23 verified against the bytes as of scoring time; two (drive3.sh, rerun_js.sh) mismatch and one superseded f_js.sh line is unverifiable because its original is absent (per the original run's `protocol/VERSIONS.md` - the freeze file, that log and the runner logs are all in the absent 2A archive, not this checkpoint). The chronology is reconstructed from runner logs, not independently verified; the byte integrity of the scored inputs was verified against registry sha512 for all 126/126 packages by the fresh measurement (the integrity record, hash-locked with the cohort), not by the original record. Timeouts are excluded and reported; only counts inside Wilson intervals are quoted.

njsscan 1.0.1. We score four stages: A, any alert; L, an alert file overlapping the adjudicated claim locus; C, an alert whose category is compatible with the claim; S, adjudication-assisted expression localisation - a reviewer judges whether the flagged expression is the one the claim names, not semantic detection by the tool. 126/126 complete and scorable (locus overrides audited: U36 3 files, U35 6 files). A = 68; L = 41/126 (Wilson 95% CI [25.0%, 41.1%]); C definite = 24. S clear (provisional, non-blind, single reviewer, post-hoc) = 17/126 (CI [8.6%, 20.5%]); because there is no second labelling pass we report explicit bounds instead of one number: strict (clear only) 17; plus the 4 ambiguous 21; partial-inclusive (plus the 3 partial) 24. A fresh measurement of this pilot was made on 2026-10-26 with re-implemented scoring code and freshly fetched tarballs: it is not a recovery of the original run, whose per-package lists are lost, and its agreement with the original aggregates is descriptive, not validation. It finds A and C at the same aggregate counts (68 and 24), L within two packages (39 against 41; the difference is hypothesised to come from the rebuilt locus-extraction heuristic, its cause not established), and it labels all 24 C candidates clear under a non-blind single-adjudicator pass - the top of the bounds quoted above; which pass is right is not resolved. The figures above remain those reported by the original scoring record, whose archive is not part of this checkpoint.

JScamd v3 (Zenodo 20567432, md5 recorded). 120/126 complete; six incomplete (f004 exited 137 twice, root cause unresolved and no host logs survive; five timed out at 110s - the original record's list; the fresh measurement's incomplete list differs environmentally, its A/L/C/S aggregates match, see the results note in the research archive). A = 10; L = 10/120 (CI [4.6%, 14.7%], bounds [10, 16]); at the automated category screen (C), one candidate survives (U43: an ECB alert whose category is compatible with the adjudicated claim); at the S stage that candidate is labelled unknown - the alert sits on a minified CryptoJS AES/DES line while the adjudicated claim is SM4 ECB on encryptData_ECB, and minification makes the mapping undecidable - recorded as unknown, not no_match. The two stages are reported separately because C is a heuristic candidate list, not evidence: the C stage contributes one candidate and zero confirmed category matches; the S stage labels it unknown; S clear = 0; bounds [0, 7].

A fixed-budget re-run of the six incompletes on a second machine reproduced all six non-completions: five timed out at 110 s, one was killed for memory in one attempt and aborted in V8 on retry. Non-completion is resource-bounded and is not evidence of no detection. Ablations on self-owned variant copies (never pooled with the fixed-protocol record) point at the scanner's serialize-and-re-parse step for the memory failure on one input; no variant was run to completion, so that explanation is supported for that input only, and the original exit-137 cause stays undetermined.

Limitations. This is an exploratory file-level measurement: no oracle- eligibility linkage, and the category-semantic mapping remains human review. The pilot checks a scoring record against the shipped bytes; it is not a second independent adjudication and no kappa is claimed. A targeted source-level re-review (against registry bytes, without re-execution) supports the "clear" labels of the three exceptional packages and the two unusual U35 claims, but the three packages' oracle executions remain open, and unknown is not no_match. The pilot is single-reviewer and its S labels are provisional pending a second pass.

Four rows answer our letters and one answers a public question; none is an independent re-grading of our rows.

Table 20. Maintainer-side responses recorded as of 2026-10-28; no further responses were recorded through 2026-11-01 (the maintainer's own statements and public records, not our grading). Names of packages whose disclosure window is open are withheld in this preprint.

finding	channel	maintainer response	public record
F-0067 (npm, L4)	email; maintainer reply the same day	two fix releases; our three recorded checks re-run against fresh installs and pass on the 2.0.0 line (1.16.0 fixes one of three)	maintainer-published GitHub security advisory (2026-10-28); CVE not yet assigned
L3 npm package	email, then GitHub private report	fix promised	advisory drafted privately, not public
F-0032 member (npm)	email	package unpublished and unmaintained	npm registry returns 404, verified independently
L3 npm package (second)	email	agrees a publication date 14 days after its next release; asked for a package-specific technical description first	release published 2026-10-28; date pending confirmation
F-0009 (Bouncy Castle)	public question	identified as known issue #218, fixed in 1.59	[34]